第 2 章 杂项数学（Miscellaneous Math）

《Fundamentals of Computer Graphics》第 5 版 • 中文译本

完整合订版：第 2.1—2.12.1 节、常见问题、注释和全部 15 道习题；保留原书数学公式与插图。

计算机图形学中的许多工作，不过是把数学表达直接转换为程序代码。数学表达越简洁，得到的代码通常也越简洁。因此，本书会反复强调：针对具体问题，应当使用恰当的数学工具。

本章回顾高中和大学数学中的若干工具，主要供读者查阅，而不是作为循序渐进的教程。这里的主题看起来可能有些零散，事实也确实如此。我们选取它们，是因为它们在常规数学课程中不那么常见、对图形学至关重要，或者通常没有从几何角度加以讨论。

除了统一本书使用的数学记号，本章还特别强调一些标准本科课程有时一带而过的内容，例如三角形的重心坐标。本章并不打算给出严格的数学论证，而是侧重直觉和几何解释。线性代数将留到第 6 章，在介绍变换矩阵之前集中讨论。

建议读者先浏览本章，熟悉涵盖的主题，遇到需要时再回来查阅。章末习题也有助于判断哪些知识需要重新温习。

2.1 集合与映射（Sets and Mappings）

映射（mapping），也称函数（function），是数学与程序设计的基础。数学中的映射与程序中的函数相似：它接收某种类型的参数，并将它映射为（或返回）另一种特定类型的对象。在程序设计中，我们称之为“类型”；在数学中，则用“集合”来描述它们。

如果对象 a 是集合 S 的元素，就使用符号 $\in$：

$$a \in S$$

它读作“a 属于集合 S”。

给定任意两个集合 A 和 B，可以通过笛卡尔积（Cartesian product）构造第三个集合，记为 $A \times B$。集合 $A \times B$ 由所有可能的有序数对 (a, b) 构成，其中 $a \in A$、$b \in B$。我们用 A^2 简记 $A \times A$。同样可以扩展笛卡尔积，从三个集合构造全部有序三元组，也可以从任意多个集合构造任意长度的有序元组。

常用的集合包括：

- R：实数集合；
- R^+：非负实数集合（包含零）；
- R^2：二维实数平面上的有序数对；
- R^n：n 维笛卡尔空间中的点；
- Z：整数集合；

- S^2：三维空间 R^3 中位于单位球面上的点的集合。

注意：尽管 S^2 的元素嵌入三维空间，但它们都位于一个可以用两个变量参数化的曲面上。因此，我们也可以将 S^2 视为二维集合。

映射的记法使用冒号和箭头，例如：

$$f : \mathbb{R} \longrightarrow \mathbb{Z}$$

可以读作：“存在一个名为 f 的函数，它以实数为输入，并把输入映射为整数。”箭头左边的集合称为函数的定义域（domain），右边的集合称为目标集合（target）。程序员可能更习惯这样的说法：“存在一个名为 f 的函数，它接收一个实数参数，返回一个整数。”换句话说，上述集合记法等价于常见的程序记法：

$$\text{integer f(real)} \leftrightarrow f : R \rightarrow Z$$

所以，不妨把冒号—箭头的写法理解为另一种程序设计语法。事情就是这么简单。

点 f(a) 称为 a 在映射 f 下的像（image）。对于定义域中的某个子集 A，它的像是目标集合中的一个子集，由 A 中所有点的像组成。整个定义域的像称为函数的值域（range）。

2.1.1 逆映射（Inverse Mappings）

对于映射 $f : A \rightarrow B$，有时存在逆映射 $f^{-1} : B \rightarrow A$，其定义规则为：如果 $b = f(a)$，那么 $f^{-1}(b) = a$。

只有在满足两个条件时，这一定义才行得通：第一，B 中的每个 b 都是定义域中某个点的像，也就是说，值域等于目标集合；第二，对每个 b，恰好只有一个这样的原像 a，即恰好有一个 a 满足 $f(a) = b$。满足这两个条件的映射称为双射（bijection）。

双射将 A 中的每个 a 映射到 B 中唯一的 b；反过来，对 B 中的每个 b，也恰好有一个 a 满足 $f(a) = b$（图 2.1）。

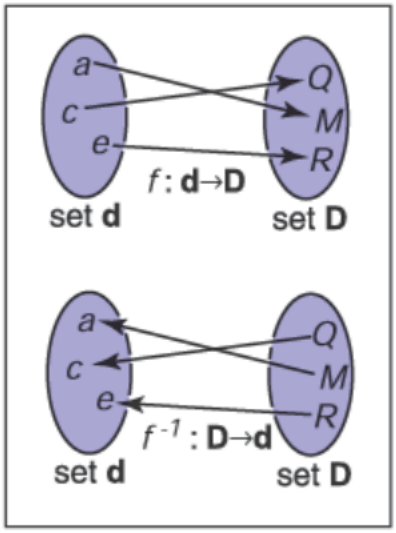


图 2.1 双射 f 与其逆映射 f^{-1}；注意 f^{-1} 也是双射。

可以把骑手和马匹之间的双射想象成：每位骑手只骑一匹马，而且每匹马恰好由一位骑手骑乘。此时存在两个函数，分别是 rider(horse) 和 horse(rider)，二者互为逆函数。不是双射的函数不存在这样的逆函数（图 2.2）。

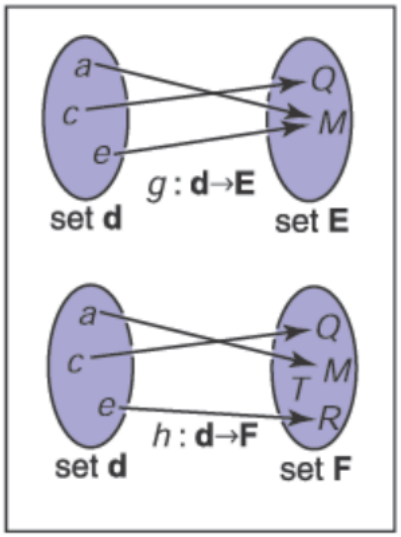


图 2.2 g 不是双射，因为两个元素映射到同一元素；h 也不是双射，因为目标集合中存在没有原像的元素。

一个双射的例子是 f : R → R，其中：

$$f(x) = x^3$$

它的逆函数为：

$$f^{-1}(x) = \sqrt[3]{x}$$

这个例子也说明，标准记号有时不够直观，因为 x 同时被用作 f 和 f^{-1} 的形式参数。换用不同的变量往往更容易理解：令 y = f(x)，同时 $x = f^{-1}(y)$，于是得到更直观的对应关系：

$$y = x^3, \qquad x = \sqrt[3]{y}$$

一个没有逆函数的例子是 sqr : R → R，其中 $\text{sqr}(x) = x^2$。原因有两个：其一，$2^2 = (-2)^2$，两个不同的输入有相同的像；其二，目标集合中的负数没有任何原像。但是，如果把定义域和目标集合都限制为 $\mathbb{R}^+$，便可以定义逆函数，此时平方根 $\sqrt{x}$ 是有效的逆函数。

2.1.2 区间（Intervals）

我们经常需要说明函数只处理某个范围内的实数。一种表示这种约束的方法就是使用区间。例如，零与一之间、不包含端点零和一的全部实数记作 (0, 1)。因为两个端点都不包含在内，所以称为开区间（open interval）。

相应地，包含两个端点的闭区间（closed interval）使用方括号表示：[0, 1]。两种括号也可以混合使用，例如 [0, 1) 包含零，却不包含一。写成区间 [a, b] 时，我们假定 a < b。区间的三种常见表示方法见图 2.3。

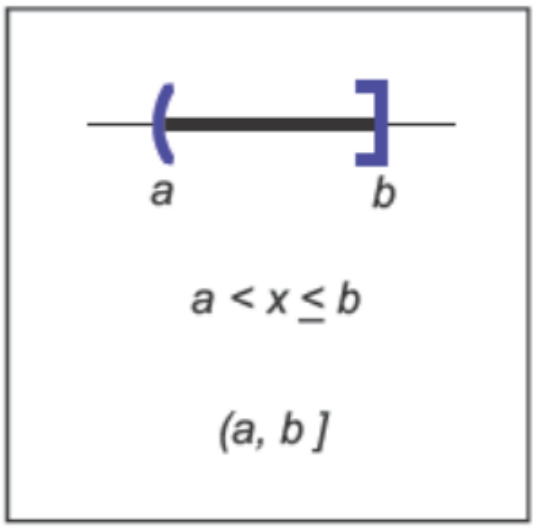


图 2.3 包含 b 而不包含 a 的区间 (a, b] 的三种等价表示。

区间的笛卡尔积也经常使用。例如，要表示三维点 x 位于单位立方体内，可以写为：

$$\mathbf{x} \in [0, 1]^3$$

区间与集合运算结合时尤其实用，常见的运算是交、并和差。两个区间的交集（intersection）是二者共有的点组成的集合，用 ∩ 表示。并集（union）用 ∪ 表示，它包含至少属于其中一个区间的点。例如：

$$[3, 5) \cap [4, 6] = [4, 5)$$
$$[3, 5) \cup [4, 6] = [3, 6]$$

与交、并不同，差集（difference）的结果取决于两个参数的顺序。差集使用减号表示，返回属于左侧集合、但不属于右侧集合的点。例如：

$$[3, 5) - [4, 6] = [3, 4)$$
$$[4, 6] - [3, 5) = [5, 6]$$

区间图能够很直观地展示这些运算（图 2.4）。

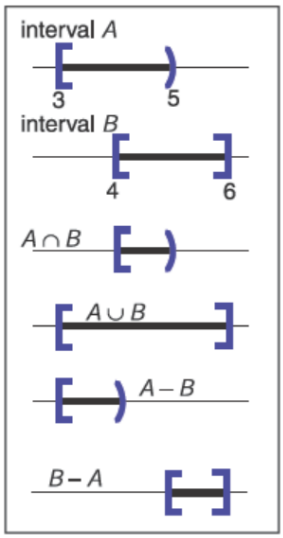


图 2.4 区间 [3, 5) 和 [4, 6] 的交、并与差运算。

2.1.3 对数（Logarithms）

虽然计算器普及之后，对数不再像过去那样常用于手工计算，但在包含指数项的方程中，对数仍然十分有用。按照定义，每个对数都有一个底数 a。“以 a 为底的 x 的对数”记为 $\log_a x$，其含义是：“要把 a 提升到多少次幂，才能得到 x？”即：

$$y = \log_a x \quad \Longleftrightarrow \quad a^y = x$$

注意，以 a 为底的对数函数与以 a 为底的指数函数互为逆函数。由这个基本定义可以得到以下关系：

$$a^{\log_a x} = x$$
$$\log_a (a^x) = x$$
$$\log_a (xy) = \log_a x + \log_a y$$
$$\log_a (x/y) = \log_a x - \log_a y$$
$$\log_a x = (\log_a b)(\log_b x)$$

在微积分中处理对数时，经常会遇到一个特殊常数 e = 2.718⋯。以 e 为底的对数称为自然对数（natural logarithm），我们采用常见的简写 ln：

$$\ln x = \log_e x$$

这里的等号也可以理解为“按定义等价”。和 π 一样，特殊常数 e 出现在数量惊人的各种问题中。

除了 e 之外，许多学科还常用另一个固定底数，并在记号中省略底数，直接写为 log x。例如，天文学家经常使用底数 10，理论计算机科学家则常用底数 2。由于计算机图形学会借鉴许多不同领域的技术，本书将避免这种省略底数的写法。

对数和指数函数的导数有助于说明为什么自然对数被称为“自然”的：

$$\frac{d}{dx}\log_a x = \frac{1}{x \ln a}$$

$$\frac{d}{dx}a^x = a^x \ln a$$

只有当 a = e 时，上面表达式中出现的常数乘子才等于一。

2.2 求解二次方程（Solving Quadratic Equations）

二次方程具有如下形式：

$$Ax^2 + Bx + C = 0$$

其中 x 是实数未知量，A、B、C 是已知常数（求解二次方程时假定 $A \neq 0$）。

在二维 x-y 坐标系中画出 $y = Ax^2 + Bx + C$，那么方程的解正好是曲线 y = 0 处的 x 坐标。因为这条曲线是抛物线，所以它可能与 x 轴没有交点、相切于一点，或者相交于两点，相应地就有零个、一个或两个实数解（图 2.5）。

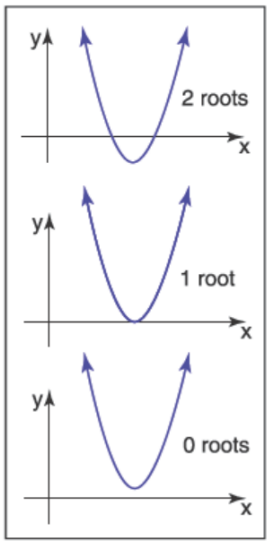


图 2.5 二次方程根的几何意义：抛物线与 x 轴的交点；可能有两个、一个或零个实根。

为了用解析方法求解，首先将方程两边除以 A：

$$x^2 + \frac{B}{A}x + \frac{C}{A} = 0$$

然后通过“配方法”（completing the square）合并各项：

$$\left(x + \frac{B}{2A}\right)^2 - \frac{B^2}{4A^2} + \frac{C}{A} = 0$$

把常数部分移到右侧，再取平方根，得到：

$$x + \frac{B}{2A} = \pm\sqrt{\frac{B^2}{4A^2} - \frac{C}{A}}$$

两边减去 B/(2A)，并将分母整理为 2A，便得到熟悉的求根公式：

$$x = \frac{-B \pm \sqrt{B^2 - 4AC}}{2A} \qquad (2.1)$$

这里的 ± 表示有两种可能的解：一种取加号，另一种取减号。因此，“3 ± 1”代表“2 或 4”。

决定实数解数量的是表达式：

$$D = B^2 - 4AC$$

它称为二次方程的判别式（discriminant）。如果 D > 0，就有两个实数解（也称根）；如果 D = 0，就有一个实数解，即“二重根”；如果 D < 0，就没有实数解。

例如，方程 $2x^2 + 6x + 4 = 0$ 的根为 x = −1 和 x = −2，而方程 $x^2 + x + 1 = 0$ 没有实数解。这两个方程的判别式分别为 D = 4 和 D = −3，与上述结论一致。

在程序中，通常应当先计算 D；如果它小于零，直接返回“没有实数根”，不要继续计算平方根。

原书脚注 1：稳健的程序实现会根据 B 的符号，选用代数上等价的表达式 $2C / (-B \pm \sqrt{(B^2 - 4AC)})$ 来计算其中一个根（见本章习题 7）。这样有助于避免浮点数相减时的精度损失。

2.3 三角学（Trigonometry）

在计算机图形学的许多场景中，都要用到基本三角学。通常并不需要特别复杂的技巧，记住基本定义就很有帮助。

2.3.1 角（Angles）

虽然我们早已习惯使用角，但为了以后将角的概念扩展到球面，有必要回到它的定义。一个角由两条半直线（从某个原点发出的无限射线）或者两个方向形成。如图 2.6 所示，两条半直线之间实际上有两种可能的角，因此必须规定如何选择。

角可定义为两条射线在单位圆上截出的弧长。一种常见约定是选择较短的那段弧，并根据两条半直线给出的顺序决定角的正负。采用这种约定时，所有角的范围都是 [−π, π]。

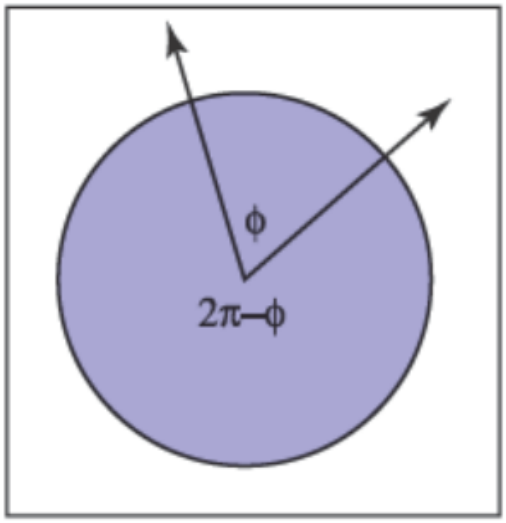


图 2.6 两条半直线将单位圆分成两段弧；可约定选择较短弧，也可按给定方向逆时针测量。

这些角的大小，就是两个方向在单位圆上所截弧段的长度。由于单位圆周长为 2π，两段可能的弧长相加等于 2π。弧长所对应的角度单位称为弧度（radian）。

另一种常见单位是度（degree）：一整圈对应 360°。因此，π 弧度等于 180°，通常记作 180°。度与弧度之间的换算关系为：

$$\text{Degrees} = \frac{180}{\pi} \text{ radians}$$

$$\text{Radians} = \frac{\pi}{180} \text{ degrees}$$

2.3.2 三角函数（Trigonometric Functions）

考虑一条直角三角形，其三条边长分别为 a、o、h；其中 h 是最长的边，位于直角的对面，称为斜边（hypotenuse）。勾股定理给出了一个重要关系：

$$a^2 + o^2 = h^2$$

图 2.7 提供了直观的证明：外部大正方形的面积为 $(a + o)^2$，四个直角三角形的总面积为 2ao，中间的小正方形面积为 h^2。由于它们恰好拼成大正方形，因此有 $2ao + h^2 = (a + o)^2$，整理之后便得到上述公式。

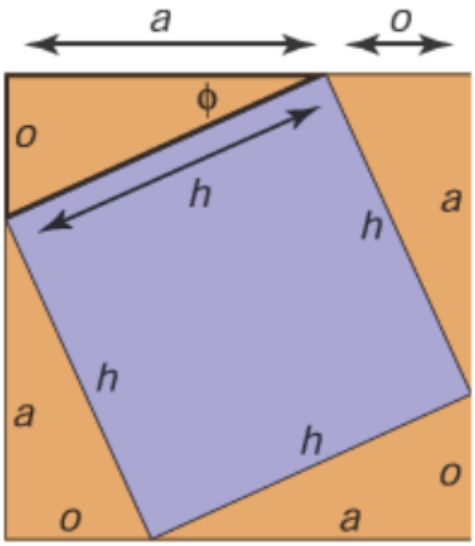


图 2.7 勾股定理的几何证明：四个三角形加中间正方形，恰好填满外部大正方形。

由此定义角 ϕ 的正弦与余弦，以及其他依据边长比值定义的三角函数：

$$\sin\phi = \frac{o}{h}, \qquad \csc\phi = \frac{h}{o}$$

$$\cos\phi = \frac{a}{h}, \qquad \sec\phi = \frac{h}{a}$$

$$\tan\phi = \frac{o}{a}, \qquad \cot\phi = \frac{a}{o}$$

这些定义使我们能够建立极坐标（polar coordinates）：一个点用它到原点的距离以及相对于 x 轴正方向的有符号角度表示（图 2.8）。

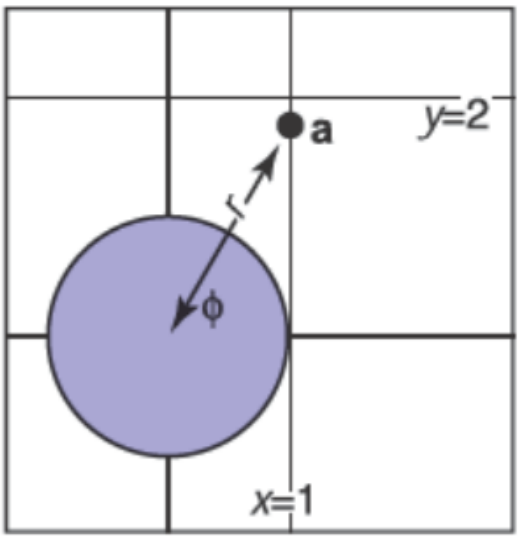


图 2.8 点 $(1, \sqrt{3})$ 的极坐标是 $(2, \pi/3)$。

注意这里采用的约定：角的范围为 $\phi \in (-\pi, \pi]$，从 x 轴正方向逆时针旋转得到正角。把逆时针方向规定为正只是人为约定，但它在图形学中广泛使用，值得牢牢记住。

三角函数是周期函数，可以接受任意角度作为参数。例如：

$$\sin A = \sin(A + 2\pi)$$

因此，当定义域为整个实数集合 R 时，这些函数都不能直接求逆。解决办法是限制标准反三角函数的输出范围；几乎所有现代数学函数库都采用了类似的标准约定（例如 Plauger，1991）。其定义域和值域为：

$$\begin{aligned} \text{asin} &: [-1, 1] \longrightarrow [-\pi/2, \pi/2] \\ \text{acos} &: [-1, 1] \longrightarrow [0, \pi] \\ \text{atan} &: \mathbb{R} \longrightarrow [-\pi/2, \pi/2] \\ \text{atan2} &: \mathbb{R}^2 \longrightarrow [-\pi, \pi] \end{aligned} \qquad (2.2)$$

译注：这里按原书式 (2.2) 保留了 atan 的闭区间写法。严格来说，实数自变量的 atan 不会取到两端的 $\pm\pi/2$，因此其数学值域是开区间 $(-\pi/2, \pi/2)$；不同程序库的 atan2 在 $\pm\pi$ 附近的端点处理也可能有所差别。

最后一个函数 atan2(s, c) 经常非常有用：传入与 sin A 成比例的值 s，以及按同一个系数缩放 cos A 得到的值 c，它就返回角 A。这里假定比例系数是正数。也可以把它理解为：返回二维笛卡尔点对应的极角（图 2.9）。

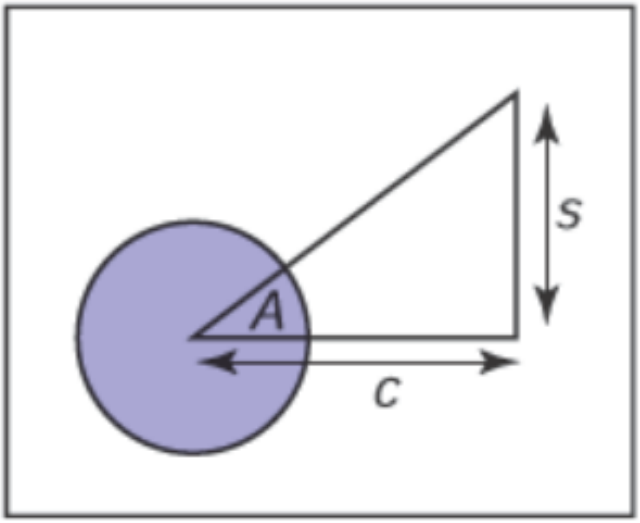


图 2.9 函数 atan2(s, c) 返回角 A，在图形学中十分有用。

2.3.3 常用恒等式（Useful Identities）

本小节不加推导地列出若干常用三角恒等式。

平移与变号恒等式：

$$\sin(-A) = -\sin A$$
$$\cos(-A) = \cos A$$
$$\tan(-A) = -\tan A$$
$$\sin(\pi/2 - A) = \cos A$$
$$\cos(\pi/2 - A) = \sin A$$
$$\tan(\pi/2 - A) = \cot A$$

勾股恒等式：

$$\sin^2 A + \cos^2 A = 1$$
$$\sec^2 A - \tan^2 A = 1$$
$$\csc^2 A - \cot^2 A = 1$$

和角与差角恒等式：

$$\sin(A + B) = \sin A\cos B + \sin B\cos A$$
$$\sin(A - B) = \sin A\cos B - \sin B\cos A$$
$$\sin(2A) = 2\sin A\cos A$$
$$\cos(A + B) = \cos A\cos B - \sin A\sin B$$
$$\cos(A - B) = \cos A\cos B + \sin A\sin B$$
$$\cos(2A) = \cos^2 A - \sin^2 A$$

$$\tan(A+B) = \frac{\tan A + \tan B}{1 - \tan A \tan B}$$

$$\tan(A-B) = \frac{\tan A - \tan B}{1 + \tan A \tan B}$$

$$\tan(2A) = \frac{2\tan A}{1 - \tan^2 A}$$

半角恒等式：

$$\sin^2(A/2) = \frac{1 - \cos A}{2}$$

$$\cos^2(A/2) = \frac{1 + \cos A}{2}$$

积化和差恒等式：

$$\sin A \sin B = -\frac{\cos(A+B) - \cos(A-B)}{2}$$

$$\sin A \cos B = \frac{\sin(A+B) + \sin(A-B)}{2}$$

$$\cos A \cos B = \frac{\cos(A+B) + \cos(A-B)}{2}$$

以下恒等式适用于任意三角形：三条边长分别为 a、b、c，它们的对角分别为 A、B、C（图 2.10）。

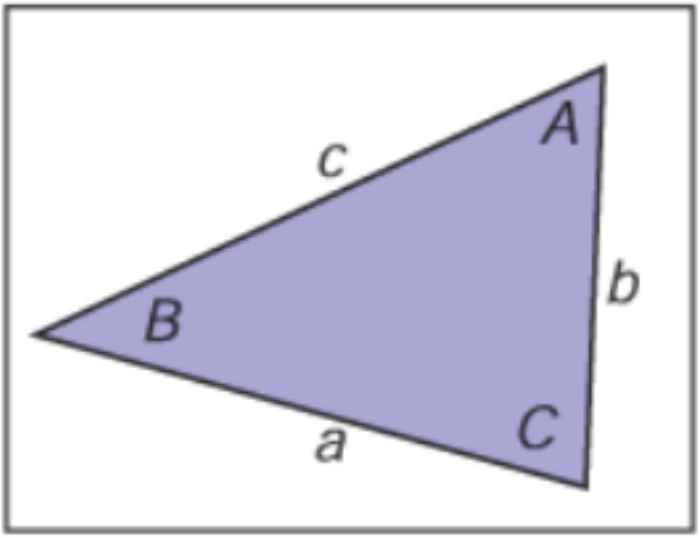


图 2.10 任意三角形的边 a、b、c 与其对角 A、B、C。

正弦定理（Law of Sines）：

$$\frac{\sin A}{a} = \frac{\sin B}{b} = \frac{\sin C}{c}$$

余弦定理（Law of Cosines）：

$$c^2 = a^2 + b^2 - 2ab\cos C$$

正切定理（Law of Tangents）：

$$\frac{a+b}{a-b} = \frac{\tan((A+B)/2)}{\tan((A-B)/2)}$$

三角形的面积还可以直接由三条边长计算：

$$\text{Area} = \frac{1}{4}\sqrt{(a+b+c)(-a+b+c)(a-b+c)(a+b-c)}$$

2.3.4 立体角与球面三角学（Solid Angles and Spherical Trigonometry）

本节前面讨论的是平面上三角形的传统三角学。三角形也可以定义在非平面曲面上，其中一种经常出现在天文学等领域的情况，是单位球面上的三角形。这类球面三角形（spherical triangle）的边，是球面上大圆的弧段；大圆是半径为一、且其平面通过球心的圆。

研究这种三角形的领域称为球面三角学（spherical trigonometry）。它在图形学中并不十分常用，但在某些问题中却至关重要。这里不详细展开，只希望读者知道这一数学领域的存在：当遇到相关问题时，可以使用球面余弦定理、球面正弦定理等许多有用的法则。例如，Arvo（1995b）关于三角形光源采样的论文，就使用了球面三角学的方法；从采样点观察，三角形光源会投影为一个球面三角形。

对于计算机图形学来说，更为核心的概念是立体角（solid angle）。普通角度可以量化这样的问题：“视野里那两根杆子相隔多远？”而立体角可以量化：“那架飞机在我的视野中占了多大一块？”

对于普通角度，我们把杆子的方向投影到单位圆上，测量它们在单位圆上对应的弧长。我们使用角度太过频繁，以至于常常忘记这个定义，因为它如今已经变得很直观。立体角其实同样简单，只是大多数人到成年以后才开始学习，所以一开始可能觉得难以理解。

对于立体角，我们把所有能够“看见”飞机的方向投影到单位球面上，测量其覆盖的球面面积。这个面积就是立体角，正如单位圆上的弧长就是平面角一样。普通角以弧度计量，围绕一整圈总计 2π（单位圆的总周长）；立体角以球面度（steradian，sr）计量，包围全部方向总计 4π（单位球面的总面积）。

> 本批译文到此结束，下一批从 2.4“向量（Vectors）”开始。第 2 章章末习题位于后续页面，将在翻译至本章末尾时完整保留和交付。

2.4 向量（Vectors）

向量（vector）描述一个长度和一个方向。我们可以方便地用箭头表示向量。只要两个向量具有相同的长度和方向，即使它们被画在不同的位置，它们也是相等的（图 2.11）。

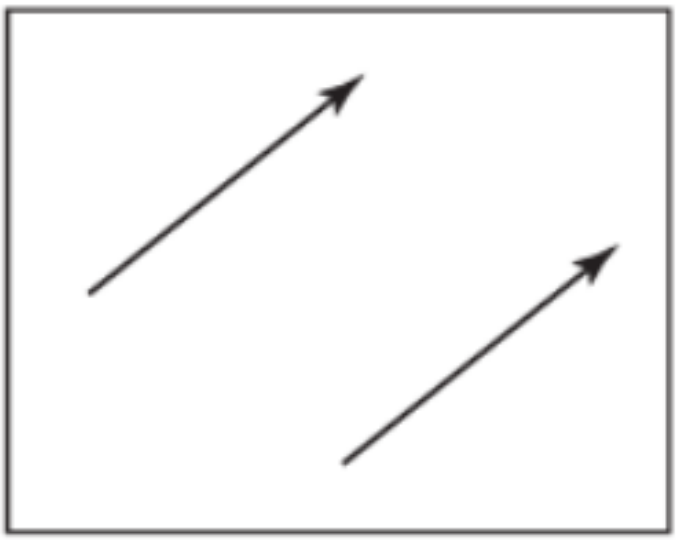


图 2.11 这两个向量具有相同的长度和方向，因此是同一个向量。

请尽可能把向量想象成箭头，而不是坐标或数字。在程序中，我们终究需要用数字来表示向量，但即使在代码里，也应该把向量当作对象来操作；只有底层向量运算才应当知道它的具体数值表示方式（DeRose，1989）。

本书使用粗体字母表示向量，例如 a 的粗体形式。向量 a 的长度记为：

$$\|\mathbf{a}\|$$

单位向量（unit vector）是长度为一的向量。零向量（zero vector）的长度为零；零向量的方向没有定义。

向量可以表示许多不同的东西。例如，它可以存储一个偏移量（offset），也称位移（displacement）。假设我们得知：“宝藏埋在秘密集合地点向东两步、向北三步的地方。”我们知道了位移，却仍不知道应该从哪里出发。

向量也能被用来存储位置（location），也就是所在地点（position）或点（point）。一个位置可以表示为它相对于另一位置的位移。通常，我们默认有一个原点（origin），其余所有位置都相对于该原点用偏移量来表示。

但要注意，位置并不是向量。正如后面会看到的，我们能够把两个向量相加；然而，除非是计算位置加权平均值时的中间步骤，否则把两个位置直接相加通常没有意义（Goldman，1985）。把两个位移相加则完全合理，这也是位移可以用向量表示的原因之一。

这里强调的是：一个位置并不等同于一个偏移量；位置是相对于某个特定原点的偏移量。脱离这个原点，偏移量本身并不能确定位置。

2.4.1 向量运算（Vector Operations）

向量拥有大多数我们熟悉的、与实数相似的算术运算。两个向量相等，当且仅当它们的长度和方向都相同。

两个向量按照平行四边形法则（parallelogram rule）相加。具体做法是：把其中一个向量的尾端接在另一个向量的箭头端点上（图 2.12）。从第一个向量的尾端指向第二个向量箭头端点、使三角形闭合的向量，就是它们的和。无论按哪一种顺序相加，都能构造出同一个平行四边形，因此向量加法满足交换律：

$$\mathbf{a} + \mathbf{b} = \mathbf{b} + \mathbf{a}$$

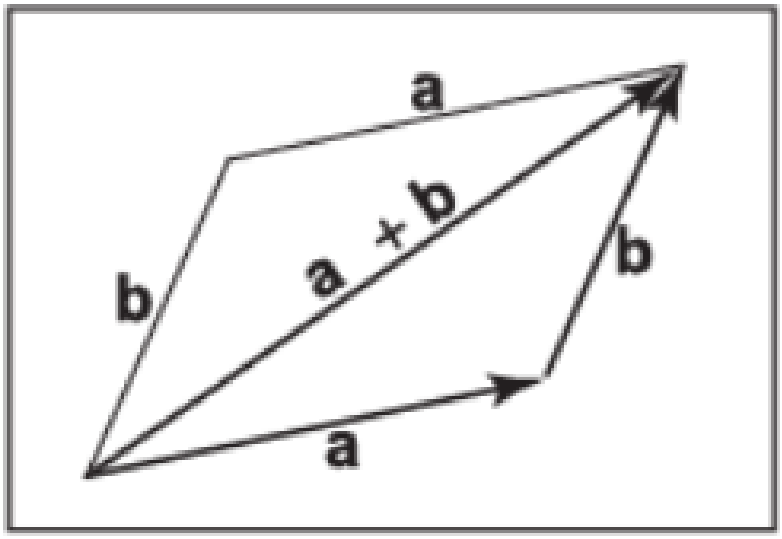


图 2.12 把两个向量首尾相接即可求和；交换相加顺序，得到的向量不变。

注意，平行四边形法则只是把我们对位移的直觉形式化了。想象先沿一个向量从尾部走到箭头，再沿另一个向量行走；总位移恰好是平行四边形的对角线。

向量还有一元负号运算：−a 表示一个与 a 长度相同、但方向相反的向量（图 2.13）。于是也可以定义向量减法：

$$\mathbf{b} - \mathbf{a} = -\mathbf{a} + \mathbf{b}$$

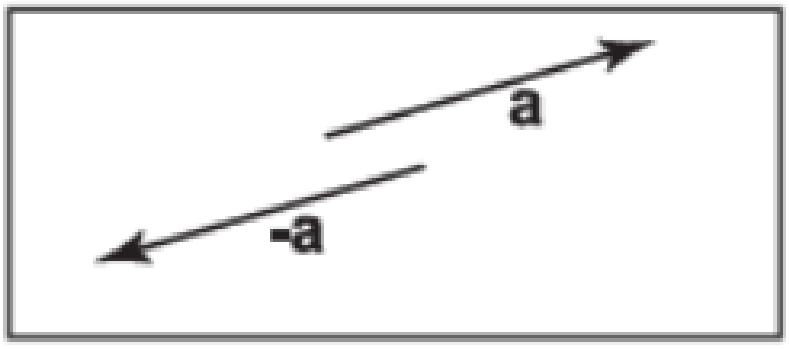


图 2.13 向量 −a 与 a 长度相同，方向相反。

我们同样可以借助平行四边形直观地理解向量减法（图 2.14）。例如：

$$\mathbf{a} + (\mathbf{b} - \mathbf{a}) = \mathbf{b}$$

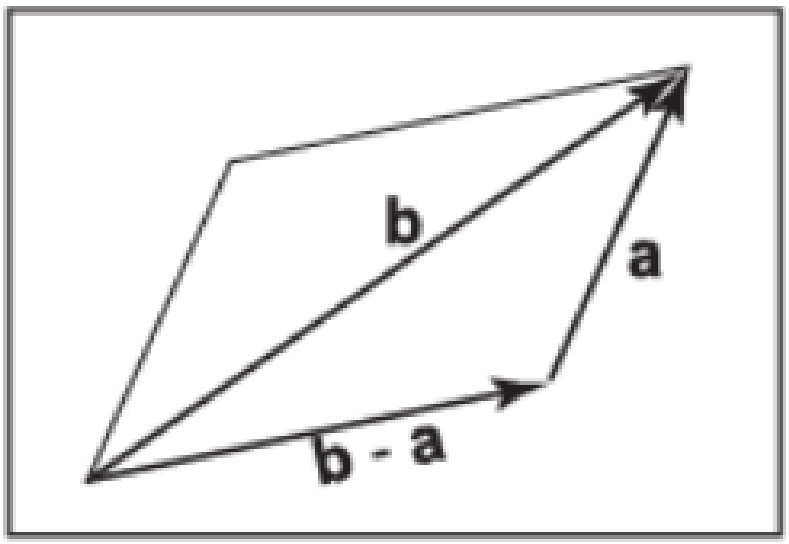


图 2.14 向量减法等价于把第二个向量取反后进行向量加法。

向量也可以相乘。事实上，涉及向量的乘积有好几种。首先，我们可以用一个实数 k 乘向量，对向量进行缩放（scale）。这会按照相应倍数改变向量的长度；当 k 为正时，方向保持不变。例如，3.5a 与 a 方向相同，但长度是 a 的 3.5 倍。

本节稍后将讨论两种涉及两个向量的乘积：点积（dot product）和叉积（cross product）。另有一种涉及三个向量的运算，即行列式（determinant），将在第 6 章讨论。

2.4.2 向量的笛卡尔坐标（Cartesian Coordinates of a Vector）

二维向量可以写成任意两个非零、且互不平行的向量的线性组合。这两个向量具有的性质称为线性无关（linear independence）。

两个线性无关的向量构成一个二维基（basis），因此又称基向量（basis vectors）。例如，向量 c 可以表示为两个基向量 a 和 b 的组合（图 2.15）：

$$\mathbf{c} = a_c\mathbf{a} + b_c\mathbf{b} \qquad (2.3)$$

注意，这里的权重 a_c 和 b_c 是唯一的。

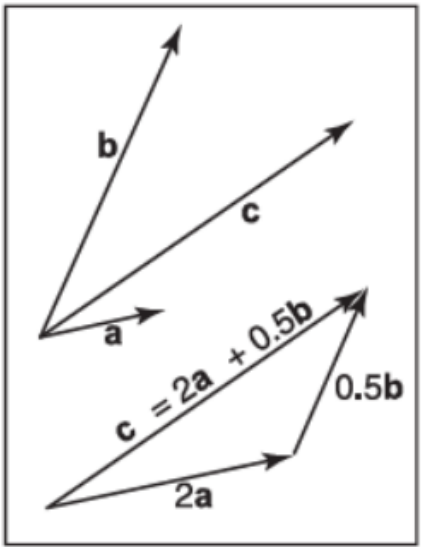


图 2.15 二维向量 c 可以表示为任意两个不平行向量 a、b 的加权和。

如果两个基向量正交（orthogonal），也就是彼此垂直，那么这个基就特别有用。如果它们同时都是单位向量，那么作用会更加明显；这种基称为标准正交基（orthonormal basis）。

假定我们已知两个这样的“特殊”向量 x 和 y，便可以在笛卡尔坐标系（Cartesian coordinate system）中表示所有其他向量，每个二维向量都用两个实数来描述。例如，向量 a 可以写成：

$$\mathbf{a} = x_a\mathbf{x} + y_a\mathbf{y}$$

其中 x_a 和 y_a 是二维向量 a 的实数笛卡尔坐标（图 2.16）。

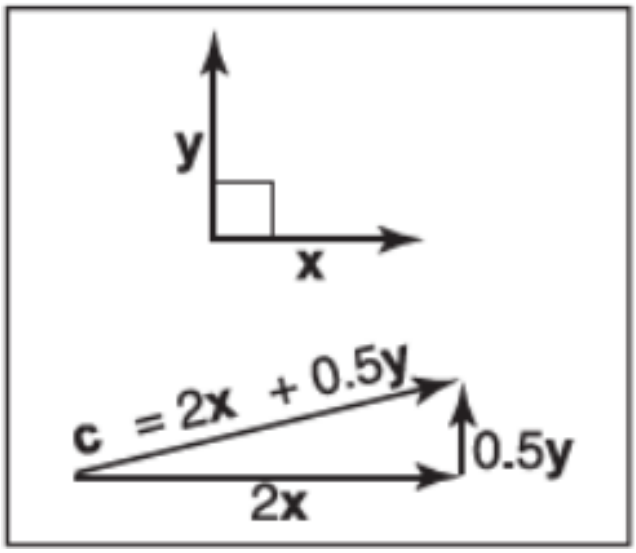


图 2.16 二维向量的笛卡尔正交基。

从概念上看，这与式 (2.3) 没有本质差别，只不过式 (2.3) 中的基向量不一定标准正交。但是，笛卡尔坐标系具有多方面优势。例如，根据勾股定理，a 的长度为：

$$\|\mathbf{a}\| = \sqrt{x_a^2 + y_a^2}$$

在笛卡尔坐标系中计算点积、叉积和向量坐标也很方便，后面将会看到这一点。

按照约定，我们通常把向量 a 的坐标写成有序数对 (x_a, y_a)，或者写成列矩阵：

$$\mathbf{a} = \begin{bmatrix} x_a \\ y_a \end{bmatrix}$$

具体采用哪种形式，取决于排版上的方便程度。我们偶尔也会把向量写成行矩阵，并通过上标 T 来表示这种写法：

$$\mathbf{a}^{\mathsf{T}} = [\, x_a \quad y_a \,]$$

我们也可以在笛卡尔坐标系中表示三维、四维乃至更高维向量。对于三维情况，可以增加一个基向量 z，使它同时垂直于 x 和 y。

2.4.3 点积（Dot Product）

两个向量相乘的最简单方法是点积（dot product）。向量 a 和 b 的点积记为 a · b。由于结果是一个标量（scalar），它也经常称为标量积（scalar product）。

点积的值与两个向量的长度，以及它们之间的夹角 ϕ 有关（图 2.17）：

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \, \|\mathbf{b}\| \cos\phi \qquad (2.4)$$

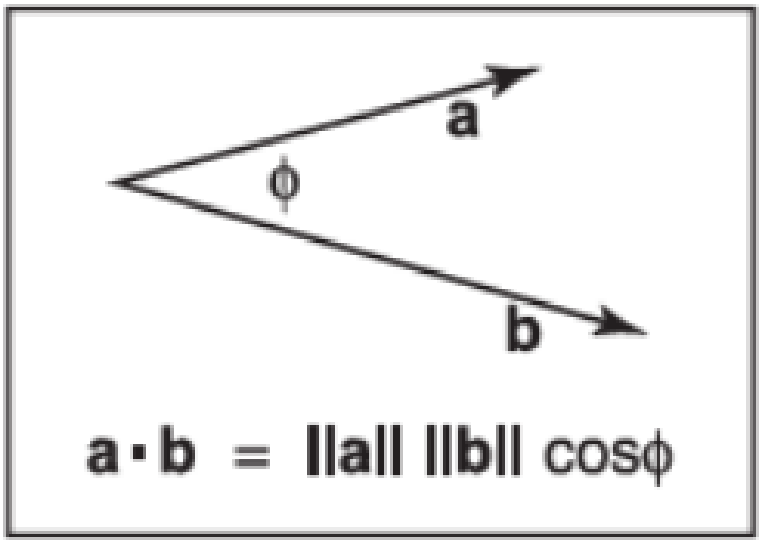


图 2.17 点积与两个向量的长度及夹角有关，是图形学最重要的公式之一。

在图形程序中，点积最常见的用途是计算两个向量夹角的余弦。

点积还可以用来求一个向量在另一个向量上的投影（projection）。这里的 a→b 表示：将向量 a 正交投影到向量 b 的方向后，得到的有符号投影长度（图 2.18）：

$$a \to b = \|\mathbf{a}\| \cos\phi = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{b}\|} \qquad (2.5)$$

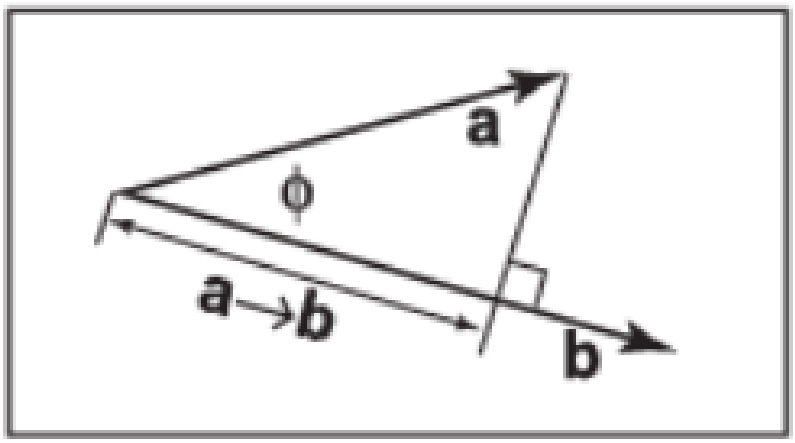


图 2.18 向量 a 在 b 方向上的投影长度可由式 (2.5) 求出。

点积具有类似于实数算术中的交换性、分配性以及与标量乘法的相容性质：

$$\mathbf{a}\cdot\mathbf{b}=\mathbf{b}\cdot\mathbf{a}$$
$$\mathbf{a}\cdot(\mathbf{b}+\mathbf{c})=\mathbf{a}\cdot\mathbf{b}+\mathbf{a}\cdot\mathbf{c}$$
$$(k\mathbf{a})\cdot\mathbf{b}=\mathbf{a}\cdot(k\mathbf{b})=k(\mathbf{a}\cdot\mathbf{b}) \qquad (2.6)$$

如果二维向量 a 和 b 使用笛卡尔坐标表示，利用 x • x = y • y = 1，以及 x • y = 0，就可以推导出点积的坐标计算公式：

$$\begin{aligned}\mathbf{a}\cdot\mathbf{b}&=(x_a\mathbf{x}+y_a\mathbf{y})\cdot(x_b\mathbf{x}+y_b\mathbf{y})\\&=x_ax_b(\mathbf{x}\cdot\mathbf{x})+x_ay_b(\mathbf{x}\cdot\mathbf{y})\\&+x_by_a(\mathbf{y}\cdot\mathbf{x})+y_ay_b(\mathbf{y}\cdot\mathbf{y})\\&=x_ax_b+y_ay_b\end{aligned}$$

对于三维向量，可以类似地得到：

$$\mathbf{a}\cdot\mathbf{b}=x_ax_b+y_ay_b+z_az_b$$

2.4.4 叉积（Cross Product）

叉积 a × b 通常只用于三维向量。更一般的叉积将在本章注释所列的参考文献中讨论。叉积返回一个三维向量，它同时垂直于两个输入向量。

结果向量的长度与两向量夹角 ϕ 的正弦有关：

$$\|\mathbf{a}\times\mathbf{b}\|=\|\mathbf{a}\|\,\|\mathbf{b}\|\sin\phi$$

叉积的模长 ||a × b||，等于以向量 a 和 b 为两条边构成的平行四边形的面积。此外，a × b 同时垂直于 a 和 b（图 2.19）。

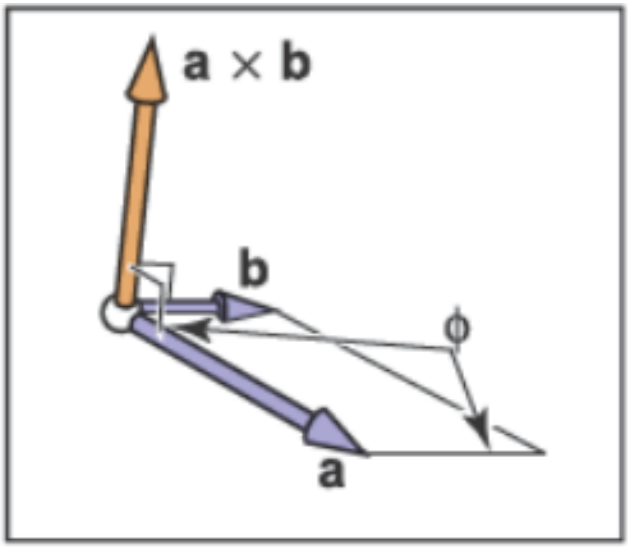


图 2.19 叉积 a × b 是同时垂直于 a 和 b 的三维向量，其长度等于图中平行四边形的面积。

注意，满足这一垂直条件的方向只有两种，即彼此相反的两个方向。按照定义，沿 x、y、z 轴正方向的单位基向量分别为：

$$\mathbf{x} = (1, 0, 0)$$
$$\mathbf{y} = (0, 1, 0)$$
$$\mathbf{z} = (0, 0, 1)$$

我们规定，x × y 必须沿 z 轴正方向或负方向。这个选择在某种程度上是任意的，但标准约定是：

$$\mathbf{z} = \mathbf{x} \times \mathbf{y}$$

三个笛卡尔单位向量之间所有可能的两两排列为：

$$\mathbf{x} \times \mathbf{y} = +\mathbf{z}$$
$$\mathbf{y} \times \mathbf{x} = -\mathbf{z}$$
$$\mathbf{y} \times \mathbf{z} = +\mathbf{x}$$
$$\mathbf{z} \times \mathbf{y} = -\mathbf{x}$$
$$\mathbf{z} \times \mathbf{x} = +\mathbf{y}$$
$$\mathbf{x} \times \mathbf{z} = -\mathbf{y}$$

根据叉积长度与 sin ϕ 的关系，同一向量与自身的叉积必为零向量，例如 x × x = 0，其他情况也一样。

还要注意，叉积不满足交换律，即 x × y ≠ y × x。

细心的读者可能发现，到目前为止，我们还没有明确指定三个笛卡尔坐标轴在空间中究竟如何排列。更具体地说：假设把 x 轴与 y 轴放在一条人行道所在的水平面上，让 x 指向东方、y 指向北方，那么 z 应当指向天空，还是指向地面？

通常约定 z 指向天空。这称为右手坐标系（right-handed coordinate system）。名称来自一个辅助记忆的方法：用右手的手掌和手指“握住”x 轴，并朝 y 轴方向旋转，拇指的方向便对应 z 轴（图 2.20）。

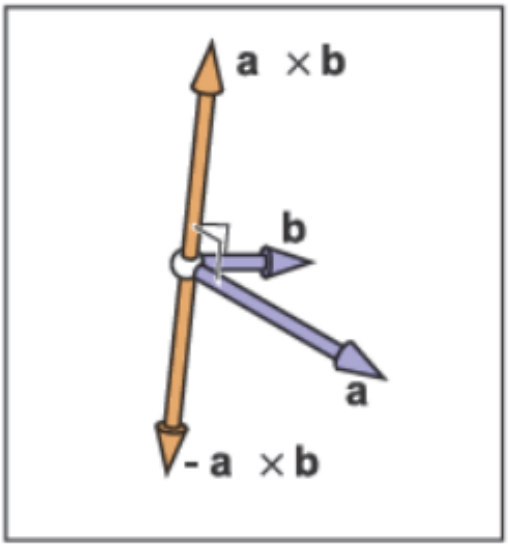


图 2.20 叉积的“右手法则”：右手从 a 转向 b，伸出的拇指指向 a × b。

叉积满足以下有用的性质：

$$\mathbf{a} \times (\mathbf{b} + \mathbf{c}) = \mathbf{a} \times \mathbf{b} + \mathbf{a} \times \mathbf{c}$$
$$\mathbf{a} \times (k\mathbf{b}) = k(\mathbf{a} \times \mathbf{b})$$

但是，根据右手法则，也必然有：

$$\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a})$$

在笛卡尔坐标系中，可以直接展开计算叉积：

$$\begin{aligned}\mathbf{a} \times \mathbf{b} &= (x_a\mathbf{x} + y_a\mathbf{y} + z_a\mathbf{z}) \times (x_b\mathbf{x} + y_b\mathbf{y} + z_b\mathbf{z}) \\ &= x_a x_b(\mathbf{x} \times \mathbf{x}) + x_a y_b(\mathbf{x} \times \mathbf{y}) + x_a z_b(\mathbf{x} \times \mathbf{z}) \\ &+ y_a x_b(\mathbf{y} \times \mathbf{x}) + y_a y_b(\mathbf{y} \times \mathbf{y}) + y_a z_b(\mathbf{y} \times \mathbf{z}) \\ &+ z_a x_b(\mathbf{z} \times \mathbf{x}) + z_a y_b(\mathbf{z} \times \mathbf{y}) + z_a z_b(\mathbf{z} \times \mathbf{z}) \\ &= (y_a z_b - z_a y_b)\mathbf{x} + (z_a x_b - x_a z_b)\mathbf{y} \\ &+ (x_a y_b - y_a x_b)\mathbf{z} \qquad (2.7)\end{aligned}$$

因此，用坐标直接表示为：

$$\mathbf{a} \times \mathbf{b} = (y_a z_b - z_a y_b,\ z_a x_b - x_a z_b,\ x_a y_b - y_a x_b) \qquad (2.8)$$

本批到式 (2.8) 结束。下一批从 2.4.5“标准正交基与坐标标架”继续，依次完成 2.4.6—2.4.8。章末习题会在到达第 2 章末尾时按原顺序完整翻译。

2.4.5 标准正交基与坐标标架（Orthonormal Bases and Coordinate Frames）

管理坐标系是几乎所有图形程序的一项核心任务，而其中的关键，是管理标准正交基（orthonormal basis）。

对于二维空间中的任意两个向量 u 和 v，只要它们互相正交（即夹角为直角），而且各自都是单位向量，它们就构成一个标准正交基。因此：

$$\|\mathbf{u}\| = \|\mathbf{v}\| = 1$$

并且：

$$\mathbf{u} \cdot \mathbf{v} = 0$$

在三维空间中，三个向量 u、v 和 w 构成标准正交基的条件为：

$$\|\mathbf{u}\| = \|\mathbf{v}\| = \|\mathbf{w}\| = 1$$

以及：

$$\mathbf{u} \cdot \mathbf{v} = \mathbf{v} \cdot \mathbf{w} = \mathbf{w} \cdot \mathbf{u} = 0$$

如果还满足：

$$\mathbf{w} = \mathbf{u} \times \mathbf{v}$$

那么这个标准正交基是右手系（right-handed）；否则它是左手系（left-handed）。

请注意，笛卡尔坐标系的规范标准正交基，只是无穷多种可能的标准正交基中的一种。它的特殊之处，是程序在进行底层数值表示时，会使用这个基以及隐含的原点。因此，基向量 x、y、z 从来不需要被显式存储，规范原点 o 也不需要显式存储。

全局模型通常就是存储在这个规范坐标系中的，因此它也常被称为全局坐标系（global coordinate system）。

不过，如果我们想使用另一个坐标系，它的原点是 p，标准正交基向量是 u、v、w，那么就确实需要把这些向量明确存储起来。这样的系统称为参考标架（frame of reference），或坐标标架（coordinate frame）。

例如，在飞行模拟器中，我们可能希望维护一个原点位于飞机机头、标准正交基方向随飞机朝向变化的坐标系。与此同时，整个模拟世界仍然有一个作为主坐标系的规范坐标系（图 2.21）。与某个特定对象（例如飞机）关联的坐标系通常称为局部坐标系（local coordinate system）。

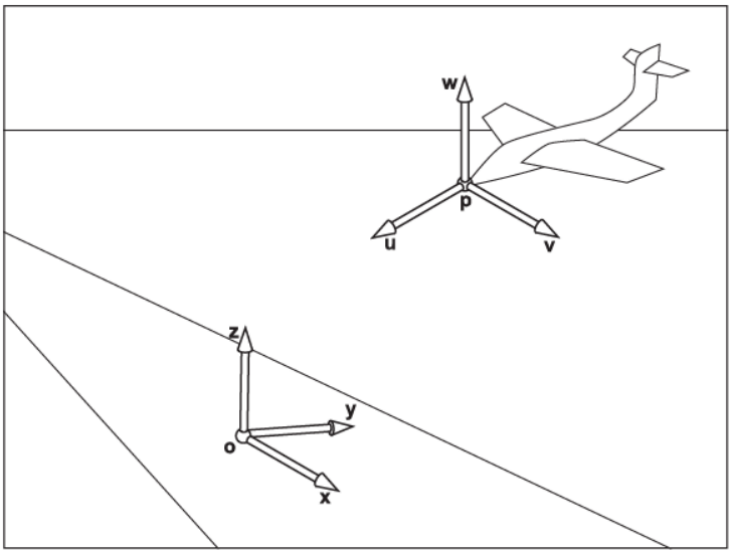


图 2.21 以 o 为原点、x、y、z 为基的主坐标系通常称为全局或世界坐标系，它不需要显式存储；飞机在 p 处的局部坐标系由 u、v、w 定义，并通过全局坐标显式存储。

在底层实现中，局部标架仍然使用规范坐标系中的坐标来存储。例如，如果 u 的坐标为 (x_u, y_u, z_u)，那么：

$$\mathbf{u} = x_u\mathbf{x} + y_u\mathbf{y} + z_u\mathbf{z}$$

一个位置还隐含了它相对于规范原点的偏移量：

$$\mathbf{p} = \mathbf{o} + x_p\mathbf{x} + y_p\mathbf{y} + z_p\mathbf{z}$$

其中，(x_p, y_p, z_p) 是位置 p 的坐标。

如果我们相对于 u-v-w 标架存储向量 a，那么实际存储的是一个三元组 (u_a, v_a, w_a)，其几何意义为：

$$\mathbf{a} = u_a\mathbf{u} + v_a\mathbf{v} + w_a\mathbf{w}$$

要把存储在 u-v-w 坐标系中的向量 a 转换为规范坐标系中的坐标，只需要记住：基向量 u、v、w 自身就以笛卡尔坐标存储。因此，只要显式计算 $u_a u + v_a v + w_a w$，得到的结果就已经采用了规范坐标系中的坐标。

反过来，假设向量 b 的坐标是以规范坐标系存储的，我们想得到它在 u-v-w 标架中的坐标，就可以利用点积：

$$u_b = \mathbf{u} \cdot \mathbf{b}$$
$$v_b = \mathbf{v} \cdot \mathbf{b}$$
$$w_b = \mathbf{w} \cdot \mathbf{b}$$

这是因为，我们知道对于某些系数 u_b、v_b、w_b，一定有：

$$u_b\mathbf{u} + v_b\mathbf{v} + w_b\mathbf{w} = \mathbf{b}$$

将两边与 u 作点积，就能单独提取出系数 u_b：

$$\begin{aligned}\mathbf{u} \cdot \mathbf{b} &= u_b(\mathbf{u} \cdot \mathbf{u}) + v_b(\mathbf{u} \cdot \mathbf{v}) + w_b(\mathbf{u} \cdot \mathbf{w}) \\ &= u_b\end{aligned}$$

之所以能够这样计算，正是因为 u、v、w 构成了标准正交基。

第 7.2.1 节和第 7.5 节将讨论如何使用矩阵来管理坐标系之间的变换。

2.4.6 从单个向量构造基（Constructing a Basis from a Single Vector）

我们经常需要一个与给定向量方向一致的标准正交基。也就是说，给定向量 a，我们希望构造出标准正交的 u、v、w，使 w 指向与 a 相同的方向（Hughes 与 Möller，1999），而 u 和 v 的具体方向则没有特别要求。

仅凭一个向量无法唯一确定整个基，但我们只需要一种稳健的方法，能够找出众多可能的基中的任意一个即可。

可以利用叉积按以下步骤完成。首先，把 w 设为沿 a 方向的单位向量：

$$\mathbf{w} = \frac{\mathbf{a}}{\|\mathbf{a}\|}$$

接下来，任选一个与 w 不共线的向量 t，利用叉积构造一个垂直于 w 的单位向量 u：

$$\mathbf{u} = \frac{\mathbf{t} \times \mathbf{w}}{\|\mathbf{t} \times \mathbf{w}\|}$$

如果 t 与 w 共线，分母就会变成零；如果它们几乎共线，计算结果的数值精度就会很差。

寻找一个与 w 足够不同的向量，有一种简单的办法：先把 t 设成 w，再把 t 中绝对值最小的分量改为 1。例如，若：

$$\mathbf{w} = (1/\sqrt{2},\ -1/\sqrt{2},\ 0)$$

那么选择：

$$\mathbf{t} = (1/\sqrt{2},\ -1/\sqrt{2},\ 1)$$

得到 w 与 u 后，只需再计算：

$$\mathbf{v} = \mathbf{w} \times \mathbf{u}$$

就完成了整个基的构造。

> 原书边栏提示：也可以按照其他顺序构造三个基向量。只需注意各次叉积中操作数的顺序，确保最终得到右手坐标基。

这种构造方法的一个应用是表面着色（surface shading）：着色时需要构建与表面法线对齐的基，但围绕法线旋转多少通常并不重要。

对于严肃的生产级代码，皮克斯（Pixar）的研究人员提出了一种特别简洁而高效的向量构造方法（Duff 等，2017）。他们提供了经过实际项目长期检验的代码。由于在业内广泛使用后并未发现明显的隐藏问题，原书鼓励读者直接采用这种实现。

2.4.7 从两个向量构造基（Constructing a Basis from Two Vectors）

上一节的方法也适用于需要控制坐标基围绕给定向量旋转角度的情形。

一个常见例子是为相机建立坐标基：其中一个向量必须与相机的观察方向对齐，但相机还可以绕这个方向旋转；这个旋转方向并非任意，必须通过某种方式指定。一旦它也确定，整个基就完全确定了。

完整指定一个标架的一种常见方法，是给出两个向量 a 和 b：前者指定 w 的方向，后者指定 v 的方向。如果已知这两个向量相互垂直，那么构造第三个向量很简单：

$$\mathbf{u} = \mathbf{b} \times \mathbf{a}$$

原书边栏提示：如果改用 u = a × b，仍然可以构造正交方向，但得到的是左手坐标系；要得到标准正交基，还需要确保输入向量已单位化。

为了保证最终得到的基确实标准正交，即使输入向量本身并不完全满足要求，我们最好采用一种与上一节“单向量”方法非常相似的步骤：

$$\mathbf{w} = \frac{\mathbf{a}}{\|\mathbf{a}\|}$$

$$\mathbf{u} = \frac{\mathbf{b} \times \mathbf{w}}{\|\mathbf{b} \times \mathbf{w}\|}$$

$$\mathbf{v} = \mathbf{w} \times \mathbf{u}$$

事实上，即使 a 和 b 并不垂直，这个过程也同样适用。在这种情况下，w 会严格指向 a 的方向，而 v 会被选为所有与 w 垂直的单位向量中，方向上最接近 b 的一个。

原书边栏提示：如果指定的 w 与 v 方向并不垂直，就无法同时完全满足两个要求。上述构造会保留 w 的指定方向，并对 v 作尽可能小的方向调整，使 v 真正垂直于 w。

如果 a 和 b 共线，这个方法就会失效。此时，b 无法帮助我们确定应该从所有垂直于 a 的方向中选择哪一个，因为 b 与这些方向全都垂直。

原书边栏思考题：如果 a 与 b 平行，上述计算会发生什么问题？

以相机定位为例（第 4.3 节）：我们希望构造一个标架，使 w 平行于相机的观察方向，而 v 指向相机的顶部。

为了让相机保持竖直，我们围绕观察方向建立坐标基，并使用“竖直向上”的方向作为参考向量，从而决定相机绕观察方向的朝向。使 v 尽可能接近竖直向上，恰好符合“把相机端正地拿着”的直观含义。

2.4.8 校正基的正交性（Squaring Up a Basis）

有时候，一个本应是标准正交的基会逐渐混入误差，给计算带来问题。例如，误差可能来自计算中的舍入，或者来自将基保存在低精度文件时造成的精度损失。

这时可以直接采用上一节的方法：使用已有的 w 和 v 向量重新构造整个基，便能得到一个标准正交、同时又与旧基相近的新基。

这种方法适用于许多应用，但并不是最理想的选择。它确实能产生精确满足正交条件的向量；如果初始基已经接近正交，结果通常也不会偏离原来的基太远。

然而，这个过程并不对称：它优先保留 w 的方向，其次考虑 v，而 u 的原始值则完全被舍弃。因此，它得到的虽然是一个与原基接近的标准正交基，却无法保证就是距离原基最近的那个标准正交基。

如果这一点不够理想，就可以使用奇异值分解（SVD，参见第 6.4.1 节），计算出一个能够保证在数学意义上最接近原始基的标准正交基。

2.5 积分（Integration）

计算机图形学中一个可能让人产生误解的地方是：这里充满了积分，因此人们可能认为，要从事图形学就必须擅长用代数方法求解积分。事实绝对不是这样。图形学中的大多数积分都无法求出解析解，因此通常需要通过数值方法求解。完全有可能在图形学领域拥有出色的职业生涯，却从未用代数方法求解过哪怕一个积分。

虽然你不需要能够通过代数方法求解积分，但必须能够读懂积分表达式，这样才能对其进行数值求解。

在一维情况下，积分通常很容易理解。例如：

$$\int_{\pi}^{2\pi} \sin(x)\,dx$$

可以读作：“计算函数 sin(x) 在 x = π 与 x = 2π 之间对应的有符号面积。”

计算机科学家可能会把其中这一部分：

$$\int_{\pi}^{2\pi} dx$$

看成一次函数调用。我们可以把这个函数叫作 integrate()。它接收两个对象：一个函数，以及一个定义域（区间）。于是，完整的调用可以写成：

```
float area = integrate(sin(), [pi,2pi]).
```

在更高阶的微积分中，我们可能开始对球面上的函数积分。对图形学来说，令人欣喜的是，仍然可以采用同样的思路来理解：

```
float area = integrate(cos(), unit-sphere)
```

这个函数内部的具体实现可能不同，但所有积分都有两个组成部分：

1. 被积函数；
2. 进行积分的定义域。

因此，通常的关键只是仔细辨认：对于手头的问题，这两部分分别是什么。这样的思维方式，与程序员阅读有时令人困惑的 API 文档、并正确调用 API 的过程颇为相似。

2.5.1 平均值与加权平均值（Averages and Weighted Averages）

积分能够求出各种量的总量，例如长度、面积、体积等。不过，积分也经常用于计算平均值。

例如，我们可以在某个区域（比如一个国家）的范围内，对地表高度进行积分，从而计算该区域对应的总体积：

```
float volume = integrate(elevation(), country)
```

但我们也可以计算平均海拔高度：

```
float averageElevation = integrate(elevation(),
    country) / integrate(1, country)
```

这基本上就是“用体积除以面积”。我们可以把它抽象为：

```
Float averageElevation = average(elevation, country)
```

我们还可以计算加权平均值（weighted average）。在这种情况下，我们引入一个权重函数，使平均过程中某些点比其他点受到更多重视。

例如，假设我们想根据温度来提高区域内某些部分的权重。这只是一个相当随意的例子，下一小节会看到更贴近图形学的例子：

```
float weightedAverageElevation =
    integrate(temperature()*elevation(),
              country) / integrate(temperature(), country)
```

遇到积分时，最好留意这种形式：许多积分实际上包含一个加权平均值，只是没有明确指出这一点。辨认出这种结构，有时能够帮助我们建立直觉。

2.5.2 立体角上的积分（Integrals over Solid Angle）

在图形学中，我们经常会遇到如下类型或与之类似的积分：

```
float shade = integrate(cos()*f*(),
                        unit-hemisphere)
```

注意，由于：

integrate(cos(), unit-hemisphere) = pi

这个积分的加权平均值形式可以直接写成：

```
float shade = integrate((1/pi)*cos()*f*(),
                        unit-hemisphere)
```

这种积分更传统的数学写法是：

$$S = \int_{\mathbf{v} \in H} \frac{1}{\pi} (\mathbf{v} \cdot \mathbf{n}) f(\mathbf{v}) (\mathbf{v} \cdot \mathbf{n}) \, d\sigma(\mathbf{v})$$

或者使用球面坐标来表示；如果要用代数方法求解这类积分，就可能采用这种写法：

$$S = \int_{\phi=0}^{2\pi} \int_{\theta=0}^{\pi} \frac{1}{\pi} (\mathbf{v} \cdot \mathbf{n}) f(\theta, \phi) \cos\theta \sin\theta \, d\theta \, d\phi$$

其中，sin θ 项是使用球面坐标时的面积修正因子。注意，在图形学中，我们很少需要把公式完整展开成这种样子。由于积分通常是通过数值方法求解的，我们往往使用不显式写出坐标的、更简单的形式。

上面的特定积分描述了一个完美反射的哑光（漫反射）表面的着色结果，它同时也是所有入射颜色的加权平均值。

这种结构非常有利于建立直觉：一个表面的颜色，通常与各种入射颜色的某种加权平均有关。

关于立体角的积分，虽然人们会使用各种各样的记号，但它们的基本结构几乎总是相同的。关键是认识到差别主要在记号上，然后把看到的记号转换成自己最熟悉的表达方式。这很像阅读伪代码！

2.6 密度函数（Density Functions）

密度函数（density function）在图形学中随处可见，例如“概率密度函数”（probability density function）。它们有时会令人意外地困惑。但是，如果我们真正掌握密度函数究竟是什么，就能更好地使用它们，并在遇到困惑时理清思路。

我们已经知道什么是函数。密度函数不过是一种返回密度值的函数。那么，什么是密度（density）？

密度是一种“每单位某种东西有多少”的量。更正式的说法是：它是一种强度量（intensive quantity）。

例如，你的体重并不是密度，而是一种广延量（extensive quantity）：它表示某种东西的总量，而不是“每单位某种东西的量”。一个人在一段固定时间（比如一年）内增加的体重，是用千克来计量的一个总量，因此它也是广延量，而不是密度。

但是，如果我们讨论的是这个人“每天”或“每小时”增加多少体重，那么它就是一种强度量，因此属于密度。

作为非密度函数的例子，考虑太阳能电池板在某一天——比如 2014 年 7 月 1 日——产生的能量。假设这一天总共产生了 120 千焦（kJ）的能量。这只是一个能量“总量”。

知道总量当然很好，但这些能量足够让我的计算机运行吗？如果我使用的是台式计算机，那么为了让它持续运行，所需要的是能量的密度，也就是能量产生或消耗的速率。

怎样才能把这一天产生的能量总量转换为能量速率呢？我们可以把这一天划分为许多时间段。例如，每段为四小时、两小时或者一小时。这样就能看到一天中不同时刻的能量产生速率存在变化；但与此同时，随着每个时间段变短，该时间段内的能量总量也在变小，如图 2.22 所示。

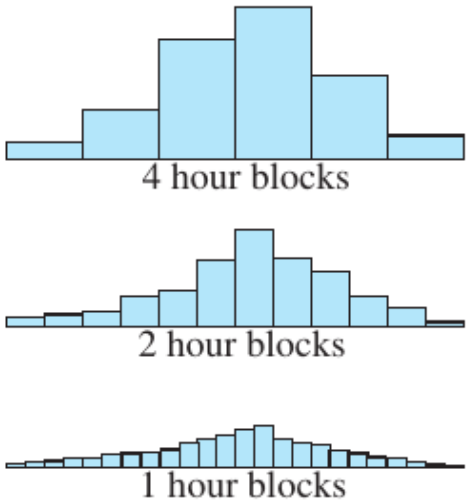


图 2.22 当统计能量产生量的时间区间不断缩短时，直方图中各柱子的高度也会降低；区间宽度趋于零时，柱高也趋于零。

如果我们把时间划分得越来越细，最终会细分到分钟和秒，就能得到越来越丰富的时间变化信息。然而，此时各个柱子的高度会变得非常小，甚至几乎什么也看不出来。

因此，我们可以根据每个柱子的宽度，重新缩放它的高度。例如：

$$\frac{30\,\text{kJ}}{0.5\,\text{h}} = 60\,\text{kJ/h}$$

如果使用这种新的“千焦每小时”（kJ/h）单位，那么随着时间区间缩短，柱子的高度就不会再自动变小，如图 2.23 所示。如果让时间段的宽度最终趋于无限小，我们就会得到一条平滑的曲线。

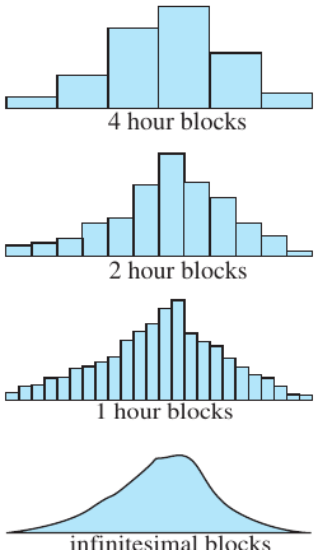


图 2.23 把每个时间段产生的能量除以该时间段的宽度之后，随着时间划分越来越细，分布的细节也越来越丰富，并逐渐形成曲线。

这条曲线就是密度函数的一个例子。有人会把它称为“能量密度”函数：密度所依赖的维度是时间。在某些语境中，它也会被称为“时间能量密度”函数。

由于这种特定的密度十分有用，也经常被讨论，所以人们给它起了一个专门的名字：功率（power）。我们不再说“每小时多少焦耳”，而是使用瓦特（watt，W）这个单位。

注意，按照约定，“瓦特”表示焦耳每秒，而不是焦耳每小时。选择哪种具体单位，通常只是为了方便；它并不会改变我们讨论的量所对应的维度。例如，有些物理量用米表示比较方便，有些用千米比较方便，还有些用纳米比较方便。

还有少数物理量，例如光的光谱辐亮度（spectral radiance），甚至会在同一个量的表达式中同时使用米与纳米。因此，如果你觉得这些单位让人困惑，也不完全是你的问题。

综合以上讨论，可以得到两点：

1. 密度总是某种比例，表示“每单位 Y 对应多少 X”，也可以简单说成“每 Y 有多少 X”，例如“每小时多少千米”。如果说成“每单位长度多少千米”听起来可能有些奇怪；但如果大家已经约定默认的长度单位，它同样是有意义的。
2. 密度函数就是返回一个密度值的函数。

密度函数本身就可以用来比较两个不同位置的相对集中程度。例如，对于前面随时间变化的能量密度函数（即功率），我们可以说：“下午两点的功率是上午九点的两倍。”

密度函数的另一种用途，是计算某一区域内的总量。例如，如果我们想求出下午两点到四点之间产生了多少能量，就只需要积分：

$$E = \int_{2\,\mathrm{pm}}^{4\,\mathrm{pm}} P(t)\,dt$$

很多积分本质上都是“对密度函数积分”，但通常并没有明确把这一点写出来。

如果我们能够辨认出，一个积分是否正在计算某个区间或区域内密度函数所对应的“总量”，就往往更容易理解这个积分的意义。

2.7 曲线与曲面（Curves and Surfaces）

2.7.1 二维隐式曲线（2D Implicit Curves）

直观地说，曲线（curve）是一组点：你可以在一张纸上不抬起笔，连续地画出这些点。一种描述曲线的常见方法是使用隐式方程（implicit equation）。二维空间中的隐式方程具有如下形式：

$$f(x, y) = 0.$$

函数 f(x,y) 返回一个实数。对于点 (x,y)，如果这个函数值为零，该点就在曲线上；如果函数值非零，该点就不在曲线上。例如，假设 f(x,y) 定义为：

$$f(x, y) = (x - x_c)^2 + (y - y_c)^2 - r^2. \qquad (2.9)$$

其中，(x_c,y_c) 是一个二维点，r 是一个非零实数。如果令 f(x,y)=0，那么满足这一等式的点就位于一个圆上：该圆以 (x_c,y_c) 为圆心、以 r 为半径。

之所以称为“隐式”方程，是因为我们无法直接从这个方程立即算出曲线上各点的坐标 (x,y)，而必须通过求解方程来确定这些点。因此，方程并没有显式地生成曲线上的点；这些点是以一种“隐含”的方式存在于方程之中的。

值得注意的是，对于所有 (x,y)，函数 f 都有对应的函数值。我们可以把 f 想象成一片地形，并将 f=0 视为海平面（图 2.24）。这样，海岸线就是隐式曲线，而 f 的值就是海拔高度。

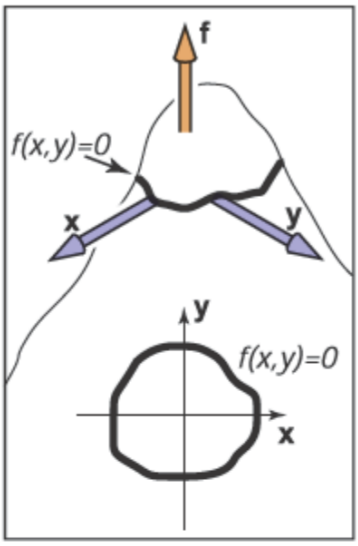


图 2.24 可以把隐式函数 f(x,y)=0 理解为一个高度场：f 是高度（上图）；高度为零的路径就是隐式曲线（下图）。

另一个需要注意的性质是：这条曲线把空间划分成 f>0、f<0 和 f=0 的区域。因此，可以通过计算 f 的值，判断某个点位于曲线的“内部”还是“外部”。请注意，对于任意常数 c，f(x,y)=c 也都定义了一条曲线；通常取 c=0 仅仅是一种约定。例如，如果 $f(x,y)=x^2+y^2-1$，那么改变 c 的值，就会得到一系列以原点为圆心的圆（图 2.25）。

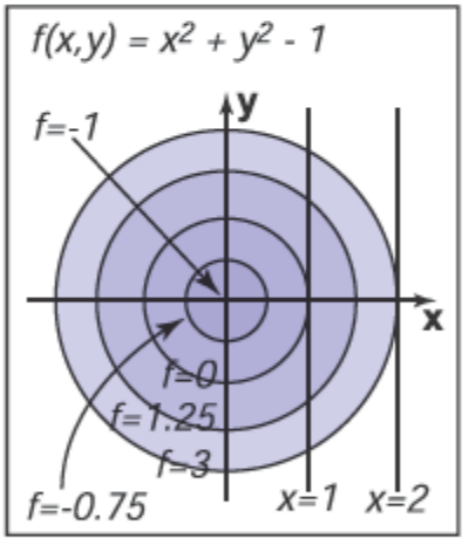


图 2.25 对隐式函数取任意常数值，都能定义一条曲线；通常约定取零。

我们可以使用向量来简化记号。令圆心向量 c=(x_c,y_c)，位置向量 p=(x,y)，那么以 c 为圆心、r 为半径的圆，可以定义为所有满足下面等式的位置向量 p 的集合：

$$(\mathbf{p}-\mathbf{c})\cdot(\mathbf{p}-\mathbf{c})-r^2=0.$$

如果把这个方程进行代数展开，就会得到式（2.9）。不过，直接从几何意义上“读懂”这个方程，会更容易看出它表示一个圆。

这个方程可以读作：“圆上的点 p 具有这样一个性质：从 c 指向 p 的向量与自身做点积，结果等于 r^2。”因为一个向量与自身的点积，恰好等于该向量长度的平方，我们也可以把它读作：“圆上的点 p 具有这样一个性质：从 c 指向 p 的向量长度的平方等于 r^2。”

更进一步，向量长度的平方也就是从 c 到 p 的距离的平方，因此我们可以得到一个等价的形式：

$$\|\mathbf{p}-\mathbf{c}\|^2-r^2=0.$$

当然，这又启发我们写出：

$$\|\mathbf{p}-\mathbf{c}\|-r=0.$$

上式可以读作：“圆上的点 p，就是与圆心 c 的距离等于 r 的那些点。”这几乎就是对圆最直接的定义。

这个例子说明：与使用 x、y 完整展开的笛卡儿坐标形式相比，向量形式的方程往往更容易体现几何意义，也更能帮助我们建立直觉。因此，在条件允许时，通常建议尽量使用向量形式。

此外，你还可以在代码中提供一个向量类；使用向量形式，代码会更简洁。以向量为基础的方程在实现时也更不容易出错：一旦你在代码中实现并调试好向量类型，那些涉及 x、y、z 分量的复制粘贴错误也就会减少。刚开始可能需要一点时间适应在方程中使用向量，但一旦熟悉，你就能从中获得很大的收益。

2.7.2 二维梯度（The 2D Gradient）

如果把函数 f(x,y) 看成一个高度场（height field），把 f(x,y) 的值看作高度，那么梯度向量（gradient vector）所指的就是上坡最陡的方向，也就是径直向上的方向。梯度向量 ∇f(x,y) 定义为：

$$\nabla f(x,y)=\left(\frac{\partial f}{\partial x},\frac{\partial f}{\partial y}\right).$$

在隐式曲线 f(x,y)=0 上的某一点处计算出的梯度向量，垂直于该点处曲线的切向量。这个垂直的向量通常称为曲线的法向量（normal vector）。此外，因为梯度指向上坡方向，所以它也指出了 f(x,y)>0 所在的区域。

在高度场的语境下，偏导数和梯度的几何意义比平时更容易看出来。假设在点 (a,b) 附近，f(x,y) 可以近似看成一个平面（图 2.26）。这时存在明确的上坡和下坡方向；与上坡方向垂直的方向，在这个平面上保持相同的高度。这个平面与高度为零的平面相交，交线也必定沿着高度不变的方向。因此，上坡和下坡的方向与交线 f(x,y)=0 垂直。

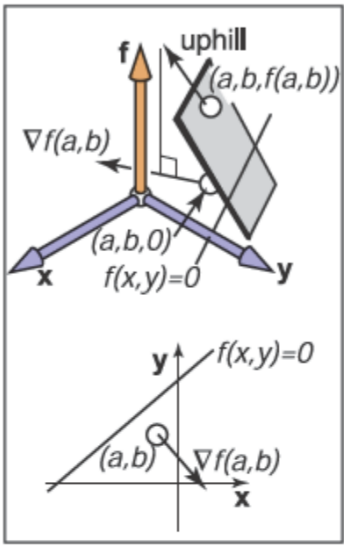


图 2.26 在 (x,y)=(a,b) 附近，高度为 f(x,y) 的曲面可以近似看作平面。梯度是“上坡”方向在高度为零的平面上的投影。

为了理解偏导数为什么与此有关，我们需要先想象它的几何意义。回想一维函数 y=g(x) 的普通导数：

$$\frac{dy}{dx} = \lim_{\Delta x \to 0} \frac{\Delta y}{\Delta x} = \lim_{\Delta x \to 0} \frac{g(x+\Delta x) - g(x)}{\Delta x}. \quad (2.10)$$

它度量的就是曲线 g 在该点处切线的斜率（图 2.27）。

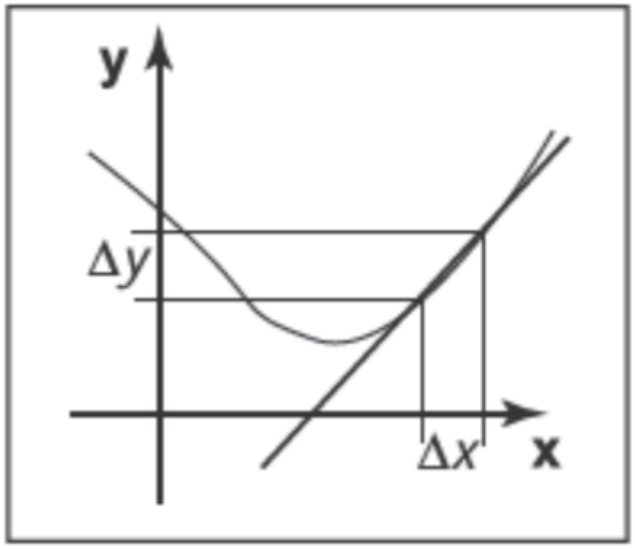


图 2.27 一维函数的导数度量该函数曲线在切点处的切线斜率。

偏导数（partial derivative）是一维导数的推广。对于二维函数 f(x,y)，不能直接像式（2.10）那样，只考虑 x 的增量来取极限，因为当 x 发生一定变化时，f 的变化可能有多种情况。不过，如果让 y 保持不变，就能定义与普通导数类似的量，这就是偏导数（图 2.28）：

$$\frac{\partial f}{\partial x} = \lim_{\Delta x \to 0} \frac{f(x+\Delta x, y) - f(x, y)}{\Delta x}.$$

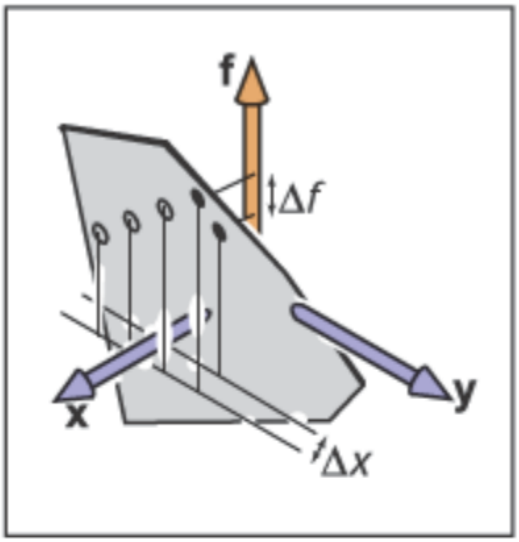


图 2.28 求函数 f 关于 x 的偏导数时，必须保持 y 不变，才能得到唯一的值，如图中的实心点所示。空心点表示 y 没有保持不变时的其他函数值。

为什么分别对 x 和 y 求偏导数，得到的恰好就是梯度向量的两个分量？同样地，从几何角度比从代数计算更容易理解。图 2.29 中，向量 a 沿着 f 不发生变化的路径移动。注意，这里仍然假设观察尺度足够小，使高度曲面 f(x,y) 在局部可以近似成平面。由图可见，向量 a=(Δx,Δy)。

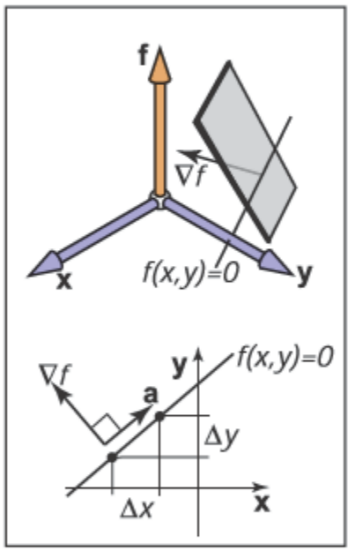


图 2.29 向量 a 指向函数 f 保持不变的方向，因此它与梯度向量 ∇f 垂直。

因为上坡方向与 a 垂直，所以它们的点积等于零：

$$(\nabla f)\cdot\mathbf{a} = (x_\nabla, y_\nabla)\cdot(x_a, y_a) = x_\nabla \Delta x + y_\nabla \Delta y = 0. \quad (2.11)$$

我们还知道，沿着 (x_a,y_a) 方向，f 的变化量也等于零：

$$\Delta f = \frac{\partial f}{\partial x}\Delta x + \frac{\partial f}{\partial y}\Delta y = \frac{\partial f}{\partial x}x_a + \frac{\partial f}{\partial y}y_a = 0.$$

对于任意两个互相垂直的向量 (x,y) 和 (x′,y′)，它们之间的夹角是 90°，因此点积为零（回想一下：点积与两个向量夹角的余弦成正比）。也就是说，xx′+yy′=0。

已知一个向量 (x,y) 时，很容易构造出与它点积为零的向量。最明显的两个例子是 (y,−x) 和 (−y,x)；你可以直接验证，它们与 (x,y) 的点积都等于零。把这个规律推广一下，对于任意非零常数 k，向量 (x,y) 都垂直于 k(y,−x)。因此，有：

$$(x_a, y_a) = k\left(\frac{\partial f}{\partial y}, -\frac{\partial f}{\partial x}\right). \quad (2.12)$$

结合式（2.11）和式（2.12），可以得到：

$$(x_\nabla, y_\nabla) = k'\left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right).$$

其中 k′ 是任意非零常数。按照定义，“上坡”意味着 f 的值沿该方向增大，所以我们希望 k′>0；约定 k′=1 就非常合适。

作为梯度的例子，考虑隐式圆 $x^2+y^2-1=0$。它的梯度向量是 (2x,2y)，这表明对于函数 $f(x,y)=x^2+y^2-1$，圆的外侧是函数值为正的区域。

请注意，梯度向量的长度会随着隐式方程所乘的系数而变化。例如，对于任意非零 A，单位圆也可以用 $Ax^2+Ay^2-A=0$ 描述。此时梯度为 (2Ax,2Ay)。这个梯度仍然垂直于圆，但它的长度由 A 决定。A>0 时，法向量指向圆外；A<0 时，法向量指向圆内。这种内外方向的翻转是合理的，因为函数值为正的区域也随之转到了圆内。

从高度场的角度看，$h=Ax^2+Ay^2-A$，圆仍然位于海拔零的位置。当 A>0 时，圆围住的是一个洼地；当 A<0 时，圆围住的则是一个凸起。A 越负，凸起的高度就越大，但 h=0 的圆本身不会改变。最大上坡方向没有改变，而坡度变得更陡了；梯度的长度正反映了坡度大小的这种变化。因此，直观地说，你可以把梯度的方向理解为“朝向上坡”，把梯度的模长理解为“上坡有多陡”。

二维隐式直线（Implicit 2D Lines）

我们熟悉的直线“斜率—截距”形式是：

$$y = mx + b. \quad (2.13)$$

它可以很容易地转换为隐式形式（图 2.30）：

$$y - mx - b = 0. \quad (2.14)$$

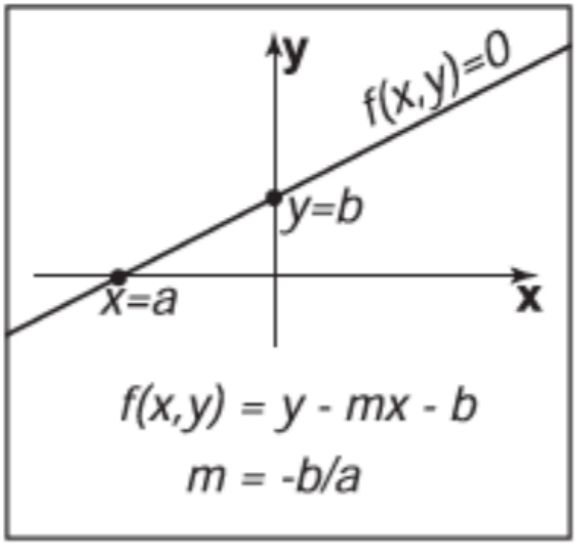


图 2.30 二维直线可以用方程 y−mx−b=0 描述。

其中 m 是“斜率”（纵向变化量与横向变化量之比），b 是直线与 y 轴交点的 y 值，通常称为 y 轴截距。直线同样把二维平面分成不同的区域，不过这里与其说“内部”和“外部”，更直观的说法或许是“上方”和“下方”。

因为隐式方程乘以任意非零常数，并不会改变使它等于零的点，所以对任意非零 k，kf(x,y)=0 表示的都是同一条曲线。这意味着同一条直线可以有多种隐式写法，例如：

$$2y - 2mx - 2b = 0.$$

斜率—截距形式有时不太方便，因为它不能表示 x=0 这样的直线：要表示这条直线，斜率 m 必须是无穷大。为此，我们常常采用更一般的形式：

$$Ax + By + C = 0, \qquad (2.15)$$

其中 A、B、C 都是实数。

假设已知直线上的两个点 (x_0,y_0) 和 (x_1,y_1)，应该选择什么样的 A、B、C，才能表示经过这两个点的直线？因为这两个点都在直线上，它们必须满足式（2.15）：

$$Ax_0 + By_0 + C = 0,$$
$$Ax_1 + By_1 + C = 0.$$

遗憾的是，我们现在只有两个方程，却有三个未知数 A、B、C。出现这个问题，是因为隐式方程允许任意非零倍数的缩放。为了方便，我们或许可以令 C=1：

$$Ax + By + 1 = 0.$$

但这样会遇到一个与斜率无穷大类似的问题：任何经过原点的直线，都必须满足 A(0)+B(0)+1=0，而这是矛盾的。例如，一条经过原点、方向为 45° 的直线，可以写为 x−y=0，也可以写为 y−x=0，甚至可以写为 17y−17x=0，却无法写成 Ax+By+1=0 的形式。

每当遇到这样棘手的代数问题时，我们都可以尝试用几何直觉来寻找解决方法。我们已经在本节中学到了一个有用的工具：梯度。对于直线 Ax+By+C=0，其梯度向量是 (A,B)。这个向量垂直于直线（图 2.31），并指向 Ax+By+C 为正值的那一侧。

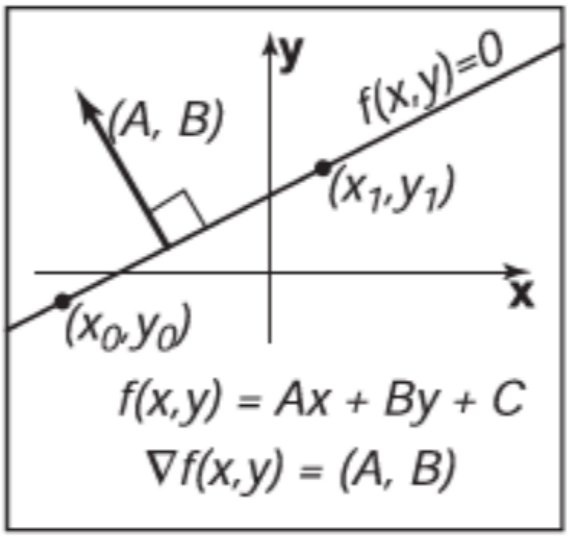


图 2.31 梯度向量 (A,B) 垂直于隐式直线 Ax+By+C=0。

已知直线上的两个点 (x_0,y_0) 和 (x_1,y_1)，连接这两点的向量就沿着直线方向。它是 (x_1-x_0,y_1-y_0)；由于它与直线平行，必定与梯度向量 (A,B) 垂直。回想一下，因为隐式方程可以任意缩放，实际上有无穷多组 (A,B,C) 能够描述同一条直线。我们只需要找到其中一组有效的系数即可。

可以先选择任意一个与 (x_1-x_0,y_1-y_0) 垂直的向量作为 (A,B)。沿用前面推导梯度时的思路，取 $(A,B)=(y_0-y_1,x_1-x_0)$ 即可。因此，经过 (x_0,y_0) 和 (x_1,y_1) 的直线方程为：

$$(y_0 - y_1)x + (x_1 - x_0)y + C = 0. \qquad (2.16)$$

现在只需要求出 C。因为 (x_0,y_0) 和 (x_1,y_1) 都在直线上，所以它们必须满足式（2.16）。将任意一个点代入，就能求出 C。若代入 (x_0,y_0)，得到 $C=x_0y_1-x_1y_0$。因此，完整的直线方程是：

$$(y_0 - y_1)x + (x_1 - x_0)y + x_0y_1 - x_1y_0 = 0. \quad (2.17)$$

这仍然只是无穷多种有效隐式方程中的一种。不过，这种形式没有除法运算，因此当两点的笛卡儿坐标都是有限数值时，不会因为除数为零而出现数值退化。

式（2.17）还有一个优点：在斜率—截距形式存在的情况下，我们总可以将与 y 无关的项移到等号右侧，再除以 y 的系数，把它转换为斜率—截距形式：

$$y = \frac{y_1 - y_0}{x_1 - x_0}x + \frac{x_1y_0 - x_0y_1}{x_1 - x_0}.$$

隐式直线方程有一个有趣的性质：它可以用于求某个点到直线的有符号距离（signed distance）。Ax+By+C 的值与该点到直线的距离成正比（图 2.32）。如图 2.33 所示，从直线上的一点沿法线方向移动向量 k(A,B)，所得的长度就是：

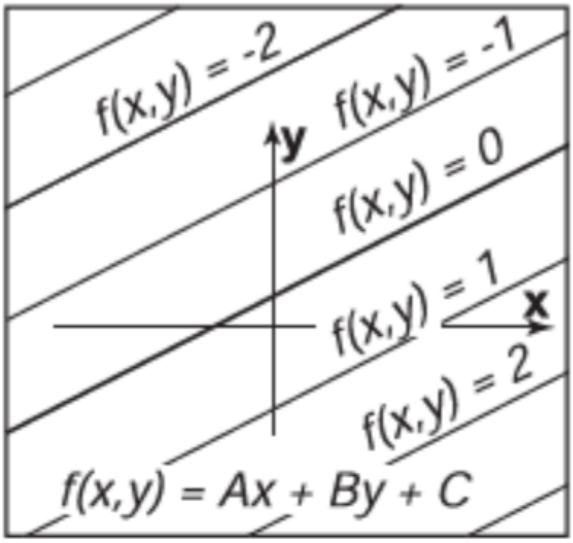


图 2.32 隐式函数 f(x,y)=Ax+By+C 的函数值，与到直线 Ax+By+C=0 的有符号距离成正比。

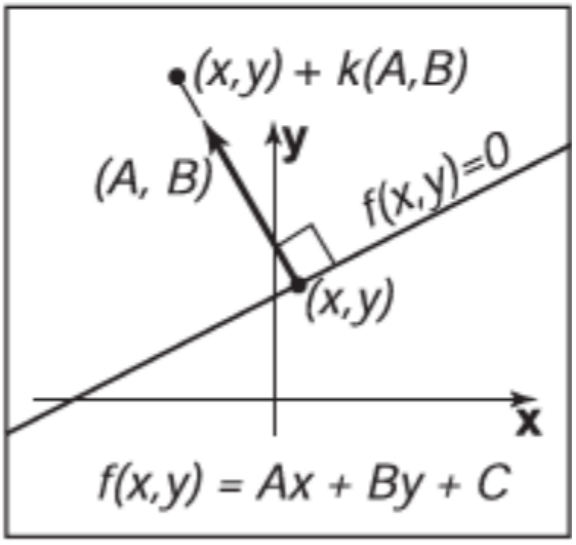


图 2.33 向量 k(A,B) 将直线上的最近点 (x,y) 与直线外的一点连接起来；距离与 k 成正比。

$$\text{distance} = k\sqrt{A^2 + B^2}. \qquad (2.18)$$

对于点 (x,y)+k(A,B)，函数 f(x,y)=Ax+By+C 的值为：

$$f(x + kA, y + kB) = Ax + kA^2 + By + kB^2 + C$$
$$= k(A^2 + B^2). \qquad (2.19)$$

这一步能够简化，是因为我们知道起点 (x,y) 位于直线上，因此 Ax+By+C=0。由式（2.18）和（2.19）可知，从直线 Ax+By+C=0 到点 (a,b) 的有符号距离为：

$$\text{Distance} = \frac{f(a,b)}{\sqrt{A^2 + B^2}}.$$

这里的“有符号距离”指：它的绝对值是普通的几何距离，但在直线的一侧取正值，在另一侧取负值。如果某个问题需要指定哪一侧为正，可以在两个等价的表示 f(x,y)=0 和 −f(x,y)=0 之间选择。

注意，如果 (A,B) 是单位向量，那么 f(a,b) 本身就是有符号距离。我们可以给式（2.17）乘上一个常数，使 (A,B) 的长度为 1，从而得到：

$$f(x,y) = \frac{y_0 - y_1}{\sqrt{(x_1 - x_0)^2 + (y_0 - y_1)^2}}x + \frac{x_1 - x_0}{\sqrt{(x_1 - x_0)^2 + (y_0 - y_1)^2}}y + \frac{x_0 y_1 - x_1 y_0}{\sqrt{(x_1 - x_0)^2 + (y_0 - y_1)^2}} = 0. \quad (2.20)$$

直接计算式（2.20）中的 f(x,y)，就能得到有符号距离；不过，要建立这个方程，需要计算一次平方根。隐式直线将在三角形光栅化中发挥重要作用（第 9.1.2 节）。其他形式的二维直线将在第 13 章讨论。

隐式二次曲线（Implicit Quadric Curves）

前面我们看到，关于 x 和 y 的线性函数 f(x,y)，通过 f(x,y)=0 可以得到一条隐式直线。如果 f 换成关于 x 和 y 的二次函数，其一般形式为：

$$Ax^2 + Bxy + Cy^2 + Dx + Ey + F = 0,$$

那么由它定义的隐式曲线就称为二次曲线（quadric）。二维二次曲线包括椭圆和双曲线，也包括抛物线、圆和直线等特殊情况。

二次曲线的例子之一，是以 (x_c,y_c) 为圆心、r 为半径的圆：

$$(x - x_c)^2 + (y - y_c)^2 - r^2 = 0.$$

另一个例子是与坐标轴对齐的椭圆：

$$\frac{(x - x_c)^2}{a^2} + \frac{(y - y_c)^2}{b^2} - 1 = 0.$$

其中 (x_c,y_c) 是椭圆的中心，a 和 b 分别是短半轴和长半轴的长度（图 2.34）。

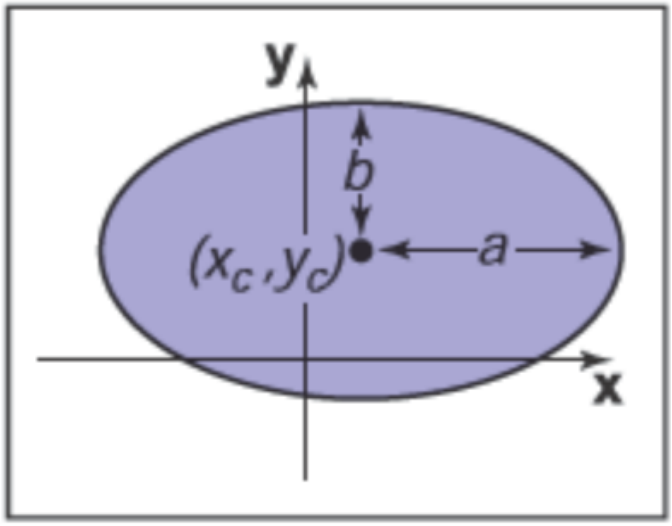


图 2.34 中心为 (x_c,y_c)、半轴长度分别为 a 和 b 的椭圆。

原书思考提示：试着在椭圆方程中令 a=b=r，并将得到的结果与圆的方程进行比较。

2.7.3 三维隐式曲面（3D Implicit Surfaces）

正如我们可以使用隐式方程来定义二维空间中的曲线一样，也可以用它来定义三维空间中的曲面。与二维情形类似，隐式方程通过隐含的方式，定义了曲面上的一组点：

$$f(x, y, z) = 0.$$

对于曲面上的任意一点 (x,y,z)，将它作为函数 f 的输入时，函数值都是零。对于不在曲面上的任意一点，函数值则是一个非零数。我们可以通过计算 f，检查某个点是否在曲面上；也可以通过 f 的正负号，判断该点位于曲面的哪一侧。不过，我们并不总能直接、显式地构造出曲面上的点。

使用向量记号，令位置向量 p=(x,y,z)，我们可以把这样的函数写为：

$$f(\mathbf{p}) = 0.$$

2.7.4 隐式曲面的法线（Surface Normal to an Implicit Surface）

曲面法线（surface normal）是一个与曲面垂直的向量。除其他用途外，光照计算就需要用到曲面法线。曲面上的不同点，可能具有不同的法向量。

就像梯度可以为二维隐式曲线提供法线一样，三维隐式曲面上某点 p 处的法线，也可以由隐式函数的梯度给出：

$$\mathbf{n} = \nabla f(\mathbf{p}) = \left(\frac{\partial f(\mathbf{p})}{\partial x}, \frac{\partial f(\mathbf{p})}{\partial y}, \frac{\partial f(\mathbf{p})}{\partial z}\right).$$

这里的道理与二维情形完全相同：梯度指向 f 增长最快的方向，因此它垂直于所有沿着曲面切线方向移动、使 f 保持不变的方向。梯度向量指向曲面上 f(p)>0 所在的一侧；在具体场景中，我们可能把这一侧称为曲面的“内部”，也可能称为“外部”。

如果某个特定的函数 f 产生的是朝向内部的梯度，而我们需要的是朝向外部的梯度，那么用 −f(p)=0 表示的曲面与用 f(p)=0 表示的曲面完全相同，但两者的梯度方向相反。换句话说：

$$-\nabla f(\mathbf{p}) = \nabla(-f(\mathbf{p})).$$

2.7.5 隐式平面（Implicit Planes）

举个例子，考虑一个经过点 a、法向量为 n 的无限延伸的平面。描述该平面的隐式方程为：

$$(\mathbf{p}-\mathbf{a})\cdot\mathbf{n}=0. \qquad (2.21)$$

注意，a 和 n 是已知量；p 则是满足该方程的任意未知点。从几何意义上看，这个方程表达的是：“从 a 指向 p 的向量与平面法线 n 垂直。”如果 p 不在平面内，那么 p–a 就不会与 n 成直角（图 2.35）。

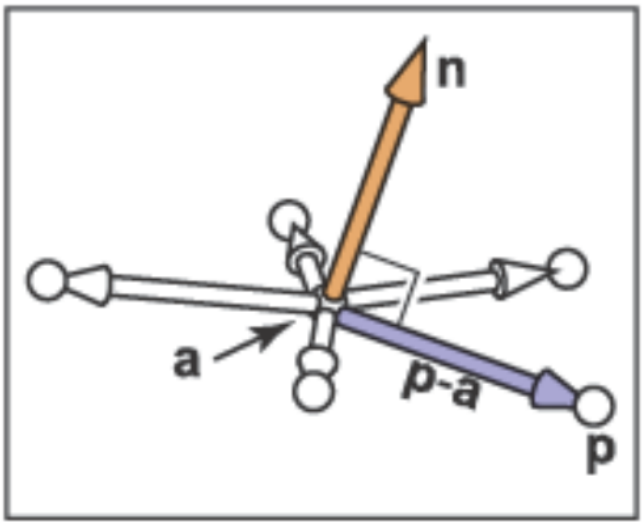


图 2.35 对于图中位于平面内的任意点 p，只要取经过点 a 的法向量 n，方程（2.21）便成立。

有时，我们希望求出经过三个点 a、b、c 的平面方程。可以通过对平面内任意两个向量做叉积，求出这个平面的法向量。这样的一种叉积写法是：

$$\mathbf{n}=(\mathbf{b}-\mathbf{a})\times(\mathbf{c}-\mathbf{a}).$$

于是，我们能够将隐式平面方程写成：

$$(\mathbf{p}-\mathbf{a})\cdot((\mathbf{b}-\mathbf{a})\times(\mathbf{c}-\mathbf{a}))=0. \quad (2.22)$$

从几何角度理解这个方程：由向量 p–a、b–a 和 c–a 所张成的平行六面体，其体积为零；也就是说，这些向量共面。当且仅当 p 与 a、b、c 在同一个平面上时，上述条件才成立。

把它完整展开为笛卡儿坐标，可以得到一个行列式的表示形式（关于行列式的更多细节见第 6.3 节）：

$$\begin{vmatrix} x-x_a & y-y_a & z-z_a \\ x_b-x_a & y_b-y_a & z_b-z_a \\ x_c-x_a & y_c-y_a & z_c-z_a \end{vmatrix}=0. \qquad (2.23)$$

式（2.23） 三阶行列式形式的隐式平面方程。

我们还可以把这个行列式进行展开（行列式展开的方法见第 6.3 节），但展开后会得到一个包含许多项、十分冗长的表达式。

式（2.22）与式（2.23）是等价的，把它们放在一起比较很有启发性。式（2.22）容易从几何角度理解，也能够写出高效的代码。此外，如果使用已经调试过的叉积和点积运算代码，那么比较容易避免那种“写错了但仍能编译，运行结果却不正确”的笔误。

式（2.23）同样容易从几何角度理解；只要实现了高效的 3×3 行列式函数，代码也会很高效。如果已经有可用的 determinant(a,b,c) 函数，实现时也不容易写错。如果把 determinant 函数重命名为 volume（体积），你的代码对其他人来说还会更容易理解。

因此，式（2.22）和式（2.23）都很容易转换成清晰的代码。相反，如果把任意一个方程完全展开成 x、y、z 各分量的冗长表达式，就很容易出现笔误。更麻烦的是，这类错误往往仍然能够通过编译，因而尤其难以排查。这是一个很好的例子：简洁的数学表达式会产生简洁的代码，臃肿的数学表达式也容易产生臃肿的代码。

三维二次曲面（3D Quadric Surfaces）

正如二维空间中，两个变量构成的二次多项式可以定义二次曲线一样，三维空间中，由 x、y、z 构成的二次多项式可以定义二次曲面（quadric surface）。例如，球面可以写成：

$$f(\mathbf{p}) = (\mathbf{p} - \mathbf{c})^2 - r^2 = 0.$$

这里，向量的平方表示该向量长度的平方。

与坐标轴对齐的椭球面可以写为：

$$f(\mathbf{p}) = \frac{(x - x_c)^2}{a^2} + \frac{(y - y_c)^2}{b^2} + \frac{(z - z_c)^2}{c^2} - 1 = 0.$$

由隐式曲面构造三维曲线（3D Curves from Implicit Surfaces）

我们或许会希望，能够直接用 f(p)=0 这样的形式，构造出一条三维隐式曲线。然而，以这种单个方程形式得到的所谓“曲线”，实际上只是某种退化曲面，在实际应用中很少有用。

一种构造三维曲线的方法，是取两个隐式方程所定义的曲面的交集，让这两个方程同时成立：

$$f(\mathbf{p}) = 0,$$
$$g(\mathbf{p}) = 0.$$

例如，两个隐式平面的交线就可以形成一条三维直线。通常，使用参数曲线（parametric curves）来描述会更加方便；后续小节将讨论这种表示方法。

2.7.6 二维参数曲线（2D Parametric Curves）

参数曲线（parametric curve）由一个单独的参数控制。我们可以把这个参数看成一种沿着曲线连续移动的“索引”。这种曲线具有如下形式：

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} g(t) \\ h(t) \end{bmatrix}.$$

这里，(x,y) 是曲线上的一个点，t 是影响曲线的参数。给定 t 的一个值，函数 g 和 h 就会确定曲线上的某个点。对于连续的 g 和 h，t 发生一个微小变化，x 和 y 也会发生微小变化。因此，随着 t 连续变化，我们会描出一条连续的曲线。

这种表示方法的一个优点是：我们可以利用参数 t，显式地构造出曲线上的点。通常，我们还可以把参数曲线写成向量形式：

$$\mathbf{p} = \mathbf{f}(t),$$

其中 f 是一个向量值函数（vector-valued function），即：

$$\mathbf{f} : \mathbb{R} \to \mathbb{R}^2.$$

这样的向量值函数能够让代码非常简洁，因此在条件允许时，应尽量使用它。

我们也可以从时间的角度来想象这条曲线：把位置看成时间的函数。曲线可以向任何地方延伸，也可能绕圈，甚至与自身相交。我们还可以想象，曲线上的点在任意位置都有某个“速度”。例如，点 p(t) 在 t=−2 附近移动得很慢，却在 t=2 到 t=3 之间移动得很快。

即使我们讨论的曲线并不真的表示一个运动中的点，在描述参数曲线时，也经常使用这种“移动的点”的说法。

二维参数直线（2D Parametric Lines）

在二维空间中，一条经过点 $p_0=(x_0,y_0)$ 和 $p_1=(x_1,y_1)$ 的参数直线，可以写为：

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x_0 + t(x_1 - x_0) \\ y_0 + t(y_1 - y_0) \end{bmatrix}.$$

因为关于 x 和 y 的方程具有十分相似的结构，所以我们可以使用向量形式，其中 p=(x,y)（图 2.36）：

$$\mathbf{p}(t) = \mathbf{p}_0 + t(\mathbf{p}_1 - \mathbf{p}_0).$$

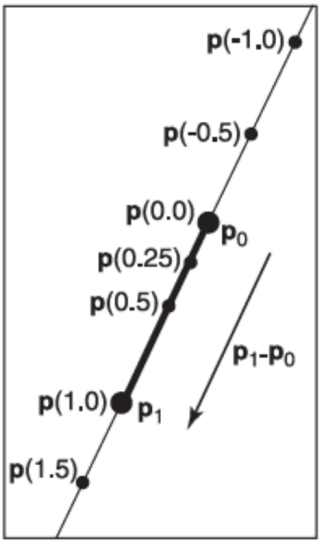


图 2.36 经过 p_0 和 p_1 的二维参数直线。其中 t∈[0,1] 所定义的线段以粗线表示。

从几何意义上，可以把这个方程读作：“从点 p_0 出发，沿着指向 p_1 的方向前进一段距离；这段距离由参数 t 决定。”这种形式的一个很好用的性质是：$p(0)=p_0$，$p(1)=p_1$。

由于点的位置会随着 t 线性变化，因此当 t 在 0 和 1 之间时，它的值也表示从 p_0 到 p_1 的距离所占的比例。当 t<0 时，点位于 p_0 的“外侧”；当 t>1 时，点位于 p_1 的“外侧”。

参数直线也可以仅用一个点 o 和一个向量 d 来表示：

$$\mathbf{p}(t) = \mathbf{o} + t\mathbf{d}.$$

当向量 d 的长度为 1 时，这条直线就采用了弧长参数化（arc-length parameterization）。这意味着，t 可以精确地度量沿着直线前进的距离。任意参数曲线原则上都可以采用弧长参数化，这显然是一种非常方便的形式；但并非所有参数曲线都能通过解析方法转换成这种形式。

二维参数圆（2D Parametric Circles）

一个圆心为 (x_c,y_c)、半径为 r 的圆，其参数形式为：

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x_c + r\cos\phi \\ y_c + r\sin\phi \end{bmatrix}.$$

为了确保曲线上的每一个点都对应唯一的参数 ϕ，我们可以限制 ϕ 的取值范围，例如 ϕ∈[0,2π)，或者 ϕ∈(−π,π]，又或者选择任意一个长度为 2π 的半开区间。

对于一个与坐标轴对齐的椭圆，我们可以分别对 x 和 y 的参数方程进行缩放，从而构造出它的参数形式：

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x_c + a\cos\phi \\ y_c + b\sin\phi \end{bmatrix}.$$

2.7.7 三维参数曲线（3D Parametric Curves）

三维参数曲线的工作方式与二维参数曲线非常相似：

$$x = f(t), \qquad y = g(t), \qquad z = h(t).$$

例如，绕 z 轴盘旋的一条螺旋线可以写成：

$$x = \cos t, \qquad y = \sin t, \qquad z = t.$$

与二维曲线一样，如果我们想控制曲线从哪里开始、在哪里结束，就需要把函数 f、g 和 h 定义在某个定义域 D 上，其中 D 是实数集的子集：

$$D \subset \mathbb{R}.$$

用向量形式，可以写为：

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \mathbf{p}(t).$$

原书边注：参数曲线是映射 p: R→R^3 的值域（range）。也就是说，让参数 t 在定义域中变化，函数 p(t) 所取到的全部位置构成这条曲线。

在本章中，我们只详细讨论三维参数直线。一般的三维参数曲线将在第 15 章中进行更深入的讨论。

三维参数直线（3D Parametric Lines）

三维参数直线可以看作二维参数直线的直接推广。例如：

$$x = 2 + 7t, \qquad y = 1 + 2t, \qquad z = 3 - 5t.$$

这种写法比较繁琐，也不太容易与代码中的变量对应。因此，我们通常使用向量形式：

$$\mathbf{p} = \mathbf{o} + t\mathbf{d},$$

其中，对于上面的例子，向量 o 和 d 分别为：

$$\mathbf{o} = (2, 1, 3),$$
$$\mathbf{d} = (7, 2, -5).$$

请注意，这与二维情形非常相似。可以这样从几何角度理解：想象这条直线经过点 o，并且与向量 d 平行。无论给 t 取什么值，都可以得到直线上的某个点 p(t)。例如，当 t=2 时：

$$\mathbf{p}(2) = (2, 1, 3) + 2(7, 2, -5) = (16, 5, -7).$$

这个基本概念与二维参数直线完全相同（参见图 2.36）。

和二维情形一样，我们可以使用一条三维参数直线，再加上一个参数区间 $t \in [t_a, t_b]$ 来描述一条线段。

两个点 a、b 之间的线段可以写成：

$$\mathbf{p}(t) = \mathbf{a} + t(\mathbf{b} - \mathbf{a}), \qquad t \in [0, 1].$$

此时：

$$\mathbf{p}(0) = \mathbf{a}, \qquad \mathbf{p}(1) = \mathbf{b},$$

并且：

$$\mathbf{p}(0.5) = \frac{\mathbf{a}+\mathbf{b}}{2},$$

即 a 与 b 之间线段的中点。

射线（ray，也称半直线 half-line）则是参数范围为半开区间的三维参数直线，其参数范围通常是 [0,∞)。

从现在开始，我们会把直线、线段和射线都简称为“射线”（ray）。严格来说，这种称呼并不精确，但它符合常见用法，也能使后面的讨论更简单。

2.7.8 三维参数曲面（3D Parametric Surfaces）

我们可以使用参数化的方法定义三维空间中的曲面，其方式与定义曲线非常相似。区别在于，曲面具有二维的面积，所以我们需要两个参数来描述它。这类曲面具有如下形式：

$$x = f(u,v), \qquad y = g(u,v), \qquad z = h(u,v).$$

或者写成向量形式：

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \mathbf{p}(u,v).$$

原书边注：参数曲面是向量值函数 p: $R^2 \to R^3$ 的值域（range）。

例 1 地球表面的一个点可以用经度与纬度这两个参数来描述。如果把坐标原点设在地球中心，并令 r 表示地球半径，那么使用一个以原点为中心的球面坐标系（图 2.37），就能够推导出下面的参数方程：

$$\begin{aligned} x &= r\cos\phi\sin\theta, \\ y &= r\sin\phi\sin\theta, \\ z &= r\cos\theta. \end{aligned} \qquad (2.24)$$

原书边注：为了便于讨论，这里假定地球是一个完全标准的球体。

原书边注：根据你过去学习的领域，你可能觉得这里的 θ 和 φ 似乎用反了，也可能并不觉得奇怪。不同学科对这些符号的使用并不统一。在本书中，我们始终采用式（2.24）及图 2.37 中的 θ、φ 定义。

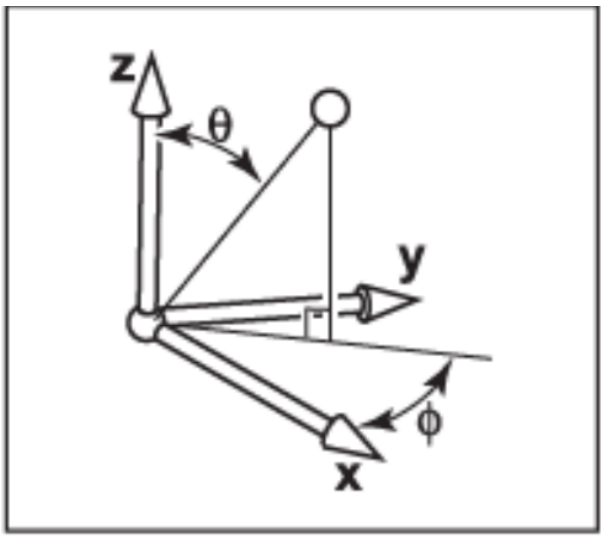


图 2.37 球面坐标系中的几何关系。

理想情况下，我们希望用向量形式来写出这个方程。不过，对于这种特定的参数形式，这样做并不可行。

我们还希望，对于给定的三维坐标 (x,y,z)，能够反过来求出参数 (θ,ϕ)。如果假设 ϕ∈(−π,π]，那么利用式（2.2）介绍的 atan2 函数，就可以很容易地做到：

$$\theta = \arccos\left(\frac{z}{\sqrt{x^2+y^2+z^2}}\right),$$
$$\phi = \text{atan2}(y, x). \qquad (2.25)$$

对于隐式曲面，我们已经知道，标量函数 f 的导数能够给出曲面法线。对于参数曲面，位置函数 p 的导数同样能够告诉我们有关曲面几何性质的信息。

考虑函数：

$$\mathbf{q}(t) = \mathbf{p}(t, v_0).$$

这个函数通过让第一个参数 u 发生变化、同时把第二个参数 v 固定为 v_0，定义出一条参数曲线。这条曲线称为等参数曲线（isoparametric curve），有时也简称为“isoparm”。它位于原来的参数曲面上。

对 q 求导，会得到一个与这条曲线相切的向量；因为这条曲线位于曲面上，所以它的切向量 q′ 也位于该曲面的切向方向。由于这条曲线是通过改变 p 的一个自变量得到的，因此 q′ 就是 p 关于 u 的偏导数，我们把它记为 p_u。

类似地，如果让 u 保持不变，那么关于 v 的偏导数 p_v，就给出了另一族等参数曲线的切向量。这是曲面上的第二个切向量。

因此，对位置函数 p 求导，可以得到曲面上任意一点处的两个切向量。我们可以对这两个向量做叉积，从而找到曲面的法线：由于这两个向量都与曲面相切，它们的叉积就同时垂直于这两个切向量，因此也垂直于曲面。

叉积的右手定则提供了一种判断曲面哪一侧是正面（front）或外侧（outside）的方法。在本书中，我们约定下面这个向量指向曲面的外侧：

$$\mathbf{n} = \mathbf{p}_u \times \mathbf{p}_v.$$

2.7.9 曲线与曲面总结（Summary of Curves and Surfaces）

二维隐式曲线和三维隐式曲面，分别由具有两个或三个自变量的标量值函数（scalar-valued function）定义。这些函数的类型可以写为：

$$f: \mathbb{R}^2 \to \mathbb{R} \qquad \text{or} \qquad f: \mathbb{R}^3 \to \mathbb{R}.$$

曲线或曲面由所有使该函数值等于零的点构成：

$$S = \{\mathbf{p} \mid f(\mathbf{p}) = 0\}.$$

二维或三维参数曲线，由一个自变量的向量值函数（vector-valued function）定义。这个函数的类型可以写为：

$$\mathbf{p}: D \subset \mathbb{R} \to \mathbb{R}^2 \quad \text{or} \quad \mathbf{p}: D \subset \mathbb{R} \to \mathbb{R}^3.$$

当参数 t 遍历整个定义域 D 时，曲线便由相应的点描绘出来：

$$S = \{\mathbf{p}(t) \mid t \in D\}.$$

三维参数曲面，由一个具有两个自变量的向量值函数定义。这个函数的类型可以写为：

$$\mathbf{p}: D \subset \mathbb{R}^2 \to \mathbb{R}^3.$$

曲面由定义域中所有 (u,v) 对应的函数值（也就是空间中的位置）构成：

$$S = \{\mathbf{p}(u, v) \mid (u, v) \in D\}.$$

对于隐式曲线和隐式曲面，法向量由函数 f 的导数（也就是梯度）给出；在此基础上，通过构造一组基向量，就可以从法向量推导出切向量：对于曲线，得到一个切向量；对于曲面，则得到多个切向量。

对于参数曲线和参数曲面，位置函数 p 的导数直接给出切向量：对于曲线，得到一个切向量；对于曲面，则得到多个切向量。接着，可以从这些切向量出发，通过构造一组基向量，推导出法向量。

2.8 线性插值（Linear Interpolation）

线性插值（linear interpolation）或许是图形学中最常见的数学运算。我们已经见过一个位置线性插值的例子：在二维和三维空间中，通过插值得到一条线段。给定点 a、b 和参数 t，可以写出下面的直线方程：

$$\mathbf{p} = (1 - t)\mathbf{a} + t\mathbf{b}.$$

之所以称为“插值”，是因为当 t=0 时，p 恰好经过 a；当 t=1 时，p 恰好经过 b。之所以称为“线性”插值，是因为权重项 t 和 1−t 都是关于 t 的一次多项式。

另一个常见的线性插值问题是：在 x 轴上有一组位置 x_0、x_1、……、x_n，每个位置 x_i 都对应一个高度 y_i。我们希望构造一个连续函数 y=f(x)，在这些位置之间进行插值，使函数曲线经过每一个数据点，也就是满足：

$$f(x_i) = y_i.$$

对于线性插值，我们用直线段把这些点 (x_i,y_i) 依次连接起来。对这些线段而言，自然可以使用参数直线方程。参数 t 就是 x 在 x_i 与 x_{i+1} 之间所处位置的距离比例，因此：

$$f(x) = y_i + \frac{x - x_i}{x_{i+1} - x_i}(y_{i+1} - y_i). \qquad (2.26)$$

由于这里的权重函数是关于 x 的一次多项式，因此这种方法属于线性插值。

上面两个例子具有相同的线性插值形式：当我们从数据项 A 移动到数据项 B 时，构造一个从 0 变化到 1 的变量 t。中间的取值就是下面这个函数：

$$(1 - t)A + tB.$$

请注意，式（2.26）也具有这个形式，只需令：

$$t = \frac{x - x_i}{x_{i+1} - x_i}.$$

2.9 三角形（Triangles）

在许多图形程序中，无论是二维三角形还是三维三角形，都是最基本的建模图元。我们经常把颜色等信息附加到三角形的顶点上，再将这些信息插值到整个三角形内部。能够使这种插值简单直接的坐标系称为重心坐标（barycentric coordinates）；下面我们将从头建立这一坐标系。我们也会讨论二维三角形，因为只有先理解二维三角形，才能在二维屏幕上绘制出它们的图像。

2.9.1 二维三角形（2D Triangles）

如果一个二维三角形由二维点 a、b、c 定义，我们首先可以求出它的面积：

$$\begin{aligned}\text{Area} &= \frac{1}{2}\begin{vmatrix} x_b - x_a & x_c - x_a \\ y_b - y_a & y_c - y_a \end{vmatrix} \\ &= \frac{1}{2}\left(x_a y_b + x_b y_c + x_c y_a - x_a y_c - x_b y_a - x_c y_b\right). \qquad (2.27)\end{aligned}$$

该公式的推导见第 6.3 节。如果点 a、b、c 按逆时针顺序排列，按照此公式计算的面积为正；否则面积为负。

在图形学中，我们常希望在三角形的每个顶点处指定某种属性（例如颜色），然后在整个三角形内部平滑地插值该属性。实现的方法有很多，其中最简单的做法是使用重心坐标。我们可以将重心坐标理解为一种非正交坐标系，正如第 2.4.2 节中简要讨论过的那样。图 2.38 展示了这样一个坐标系：原点是点 a，基向量是 b−a 和 c−a。以这些基向量及原点表示，任何点 p 都可写为：

$$\mathbf{p} = \mathbf{a} + \beta(\mathbf{b} - \mathbf{a}) + \gamma(\mathbf{c} - \mathbf{a}). \qquad (2.28)$$

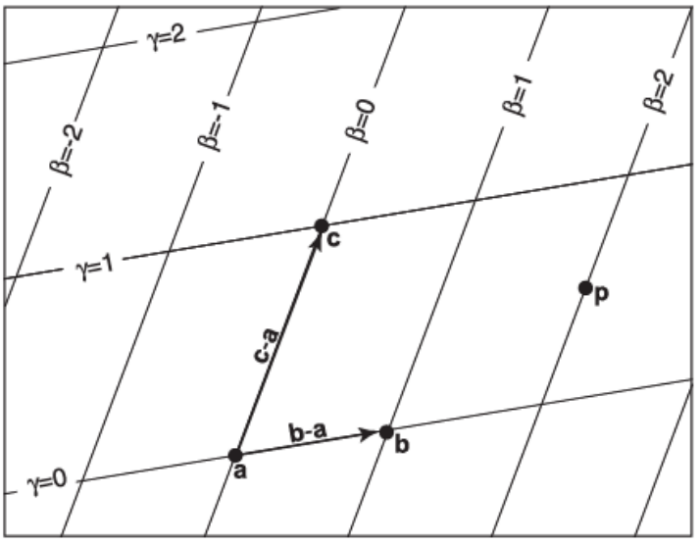


图 2.38 以顶点 a、b、c 构成的二维三角形，可以建立以 a 为原点、以 b−a 和 c−a 为基向量的非正交坐标系。在该坐标系中，点由有序数对 (β, γ) 表示。例如图中 p=(2.0, 0.5)，即 p=a+2.0(b−a)+0.5(c−a)。

注意，将式（2.28）的各项重新排列，可得到：

$$\mathbf{p} = (1 - \beta - \gamma)\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}.$$

人们通常再定义一个新变量 α，使等式在形式上更加对称：

$$\alpha \equiv 1 - \beta - \gamma,$$

于是得到：

$$\mathbf{p}(\alpha, \beta, \gamma) = \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}. \qquad (2.29)$$

同时还必须满足约束条件：

$$\alpha + \beta + \gamma = 1. \qquad (2.30)$$

重心坐标乍看之下似乎抽象而不直观，但实际上既有用又方便。可以想象这样一座城市：城市道路由两组平行街道组成，但这两组街道并不互相垂直。在这样的城市中，自然形成的地址系统本质上就相当于重心坐标，而人们很快就能习惯它。

平面上的所有点都可以用重心坐标表示。它的一项特别好的性质是：当且仅当满足下面三个条件时，点 p 才位于由 a、b、c 构成的三角形内部：

$$0 < \alpha < 1,$$
$$0 < \beta < 1,$$
$$0 < \gamma < 1.$$

如果某一个坐标为零，而另外两个坐标都在零和一之间，那么该点位于一条边上；如果两个坐标为零，则第三个坐标为一，该点就在一个顶点上。

重心坐标还有另一项很好的性质：式（2.29）实际上以平滑的方式混合了三个顶点的坐标。同样的混合系数 (α, β, γ) 也可以用于混合其他属性，例如颜色。我们将在下一章看到这一应用。

已知点 p，怎样求出它的重心坐标？一种方法是将式（2.28）写成以 β 和 γ 为未知数的线性方程组，求解后再令 α=1−β−γ。这个线性方程组是：

$$\begin{bmatrix} x_b - x_a & x_c - x_a \\ y_b - y_a & y_c - y_a \end{bmatrix} \begin{bmatrix} \beta \\ \gamma \end{bmatrix} = \begin{bmatrix} x_p - x_a \\ y_p - y_a \end{bmatrix}. \qquad (2.31)$$

虽然从代数上直接求解式（2.31）并不困难，但使用几何方法求解往往更有启发性。

重心坐标的一项几何性质是：它们分别表示从点到三角形各条边所在直线的、有符号且经过缩放的距离。图 2.39 给出了 β 的情况。回顾第 2.7.2 节，对直线方程 f(x,y)=0 代入坐标 (x,y)，所得的 f(x,y) 就是该点到直线的一个经过缩放的有符号距离。另外，如果 f(x,y)=0 表示某条特定直线，那么对于任何非零的 k，方程 kf(x,y)=0 也表示同一条直线。改变 k 可以缩放该距离，并控制直线哪一侧对应正值、哪一侧对应负值。

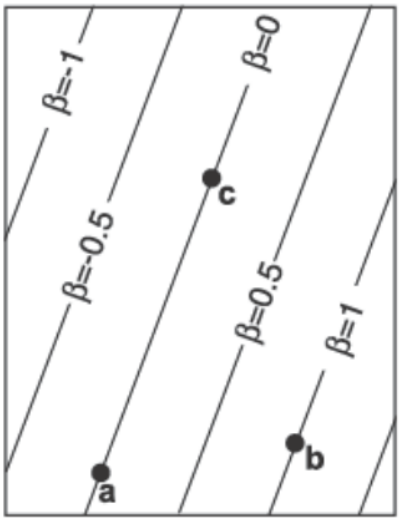


图 2.39 重心坐标 β 是点到经过 a、c 两点的直线的有符号、经过缩放的距离。

我们希望选择 k，使得 kf(x,y)=β。因为 k 只有一个未知量，我们可以施加一个约束来确定它：在点 b 处，β=1。如果穿过 a 和 c 的直线方程为 $f_{ac}(x,y)=0$，那么坐标为 (x,y) 的点 p 所对应的 β 就可写为：

$$\beta = \frac{f_{ac}(x,y)}{f_{ac}(x_b,y_b)}. \qquad (2.32)$$

用类似方法也能计算 γ 和 α。为了提高效率，通常应直接计算重心坐标中的两个分量，再利用式（2.30）求出第三个分量。

为了求出经过 p_0 和 p_1 的直线的这种“理想”形式，我们可以使用第 2.7.2 节介绍的方法，找到一个经过两个顶点的有效隐式直线方程。式（2.17）给出：

$$f_{ab}(x,y) \equiv (y_a - y_b)x + (x_b - x_a)y + x_a y_b - x_b y_a = 0.$$

请注意，$f_{ab}(x_c,y_c)$ 很可能不等于 1，因此这个方程还不是我们想要的理想形式。将其除以 $f_{ab}(x_c,y_c)$，得到：

$$\gamma = \frac{(y_a - y_b)x + (x_b - x_a)y + x_a y_b - x_b y_a}{(y_a - y_b)x_c + (x_b - x_a)y_c + x_a y_b - x_b y_a}.$$

这里出现的除法可能让人担心，因为分母有可能为零。但只要三角形的面积不是零或接近零，就不会发生这个问题。对于 α 和 β 也存在类似的公式，不过通常只需要再计算其中一个：

$$\beta = \frac{(y_a - y_c)x + (x_c - x_a)y + x_a y_c - x_c y_a}{(y_a - y_c)x_b + (x_c - x_a)y_b + x_a y_c - x_c y_a},$$

$$\alpha = 1 - \beta - \gamma.$$

计算重心坐标还有另一种方法：如图 2.40 所示，求出三个子三角形的面积 A_a、A_b 和 A_c。重心坐标遵循以下关系：

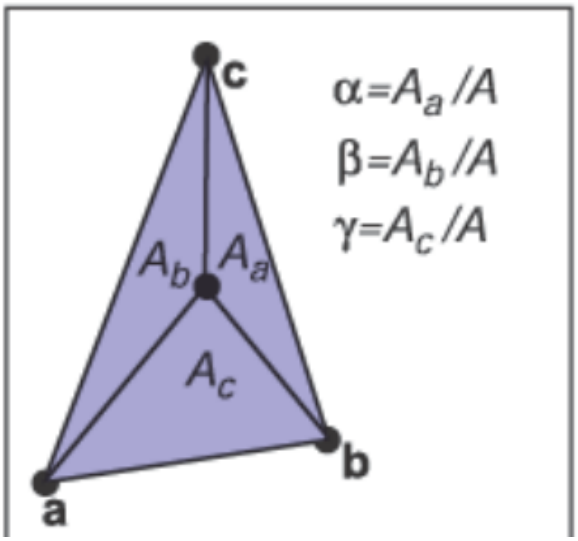


图 2.40 重心坐标与图中三个子三角形的面积成正比。

$$\alpha = \frac{A_a}{A}, \qquad \beta = \frac{A_b}{A}, \qquad \gamma = \frac{A_c}{A}. \qquad (2.33)$$

其中 A 为整个三角形的面积。注意 $A=A_a+A_b+A_c$，因此只需做两次加法，就能得到 A，而不必重新使用完整的面积公式。

只要允许面积带符号，该规则对于位于三角形外部的点同样成立。图 2.41 解释了原因。还要注意，这里所用的都是有符号面积：只要计算 A 和三个子三角形面积 A_a、A_b、A_c 时采用相同的有符号面积计算方法，就能得到正确结果。

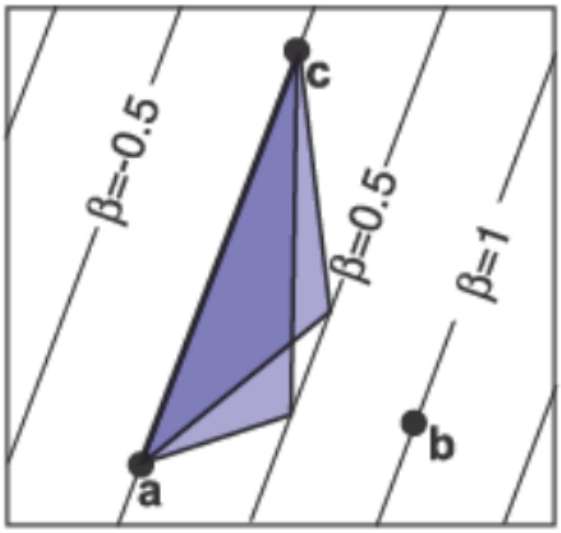


图 2.41 图示的两个三角形具有相同的底和高，因此面积相等；任何一个顶点位于 β=0.5 直线上的三角形也有相同的面积。三角形的高及其面积均与 β 成正比。

2.9.2 三维三角形（3D Triangles）

重心坐标有一个令人称道的优点：它们几乎可以自然而然地推广到三维空间。如果我们假设点 a、b、c 都是三维点，那么仍然可以使用下面的表示方式：

$$\mathbf{p} = (1 - \beta - \gamma)\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c}.$$

此时，当 β 和 γ 发生变化时，点 p 就会扫出一个平面。

三角形的法向量可以通过求位于三角形平面内的任意两个向量的叉积得到（图 2.42）。最简单的做法，是从三条边中取出两条，将它们作为向量。例如：

$$\mathbf{n} = (\mathbf{b} - \mathbf{a}) \times (\mathbf{c} - \mathbf{a}). \qquad (2.34)$$

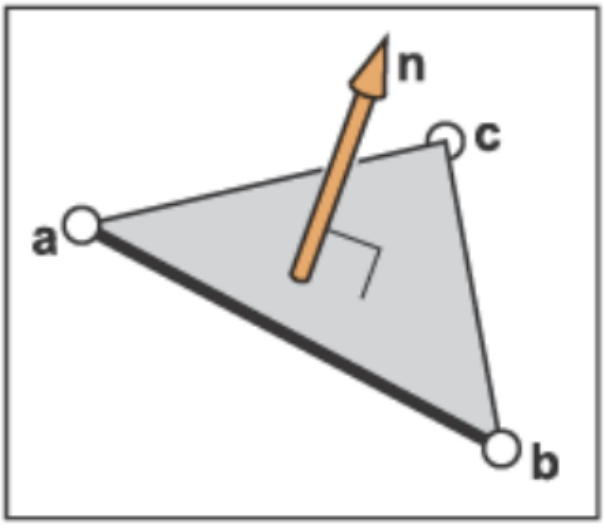


图 2.42 三角形的法向量垂直于三角形所在平面内的所有向量，因此也垂直于该三角形的各条边。

请注意，这个法向量不一定具有单位长度，而且它遵循叉积的右手定则。

三角形的面积可以由叉积向量的长度计算得到：

$$\text{area} = \frac{1}{2}\|(\mathbf{b}-\mathbf{a})\times(\mathbf{c}-\mathbf{a})\|. \qquad (2.35)$$

注意，这个面积不是有符号面积，因此不能直接用于计算重心坐标。不过我们可以观察到：如果一个三角形的顶点按顺时针顺序排列，那么它的法向量会与位于同一平面内、顶点按逆时针顺序排列的三角形的法向量指向相反的方向。

回顾点积的公式：

$$\mathbf{a}\cdot\mathbf{b} = \|\mathbf{a}\|\,\|\mathbf{b}\|\cos\phi,$$

其中 ϕ 是两个向量之间的夹角。如果 a 和 b 相互平行，那么 cos ϕ=±1，这便可以用来检验两个向量是指向同一方向还是相反方向。

结合上述性质以及式（2.33）—（2.35），我们可以推得：

$$\alpha = \frac{\mathbf{n}\cdot\mathbf{n}_a}{\|\mathbf{n}\|^2},$$
$$\beta = \frac{\mathbf{n}\cdot\mathbf{n}_b}{\|\mathbf{n}\|^2},$$
$$\gamma = \frac{\mathbf{n}\cdot\mathbf{n}_c}{\|\mathbf{n}\|^2}.$$

这里，n 是把顶点 a、b、c 代入式（2.34）得到的法向量；n_a 是把顶点 b、c、p 代入式（2.34）得到的向量，其余两个向量依此类推。也就是说：

$$\begin{aligned}\mathbf{n}_a &= (\mathbf{c}-\mathbf{b})\times(\mathbf{p}-\mathbf{b}),\\ \mathbf{n}_b &= (\mathbf{a}-\mathbf{c})\times(\mathbf{p}-\mathbf{c}), \qquad (2.36)\\ \mathbf{n}_c &= (\mathbf{b}-\mathbf{a})\times(\mathbf{p}-\mathbf{a}).\end{aligned}$$

2.10 离散概率（Discrete Probability）

概率论研究的是包含随机结果的事物；当可能出现的随机结果只有有限个时，我们讨论的就是离散概率（discrete probability）。一个经典例子是六面骰子：掷一次骰子，可能随机出现集合 {1, 2, 3, 4, 5, 6} 中的一个值，并且每种结果出现的概率相同。某个特定结果的概率，就是该结果在所有试验中出现的次数所占的比例。某件事情总会发生的概率是 1；骰子每个面的结果出现的概率都是六分之一。

关于随机性，一个比较容易令人困惑的地方，是区分“尚未发生的随机结果（或者结果已经发生，但我们还不知道它）”与“骰子已经掷出后得到的结果”。随机变量（random variable）表示一个尚不知其具体取值、但会从一组已知的可能值中依照已知概率取值的量。这里的“变量”一词来自数学，与程序设计中的“变量”直接相关。

例如，我们可以定义随机变量 X，其中“X=掷出骰子最终得到的结果”。这个变量可以使用任何符号；在数学中，通常以大写 X 表示随机变量，就像在计算机科学中，人们经常使用 i 和 j 表示循环变量一样。在计算机程序中，随机变量可以非常直接地使用：

```
int X = rand_from(1,6)
```

这里的 X 是一个尚不知道具体数值的变量，但我们知道，程序运行时，它将以各六分之一的概率得到六个值中的某一个。这与数学中的随机变量“X=骰子最终出现的点数”完全对应。

随机变量有两个经常用到的性质：期望值（expected value）和方差（variance）。随机变量 X 的期望值（有时称为数学期望，expectation）通常记作 EX 或 E(X)。也许把它称作“预期平均值”会更恰当，不过最好不要自行更改名称，否则会让熟悉标准术语的人感到困惑。它实际上就是在所有“掷出骰子”的平行宇宙中，X 所取得的值的平均数。计算方法是：每个可能结果乘以其概率，然后求和：

$$EX = \tfrac{1}{6}(1) + \tfrac{1}{6}(2) + \tfrac{1}{6}(3) + \tfrac{1}{6}(4) + \tfrac{1}{6}(5) + \tfrac{1}{6}(6) = 3.5.$$

因此，如果我们掷很多次骰子，并对结果取平均，就会“预期”得到一个接近 3.5 的数值。明明骰子只能掷出整数，却说“我预期骰子的结果是 3.5”，听起来似乎荒谬，但理解了这个概念之后就不难接受。或许这里使用的术语并不十分理想，但它已经成为各个领域广泛采用的标准用语。因此，尽量熟悉这一术语，就能顺利与其他领域的人讨论相关问题。

期望值告诉我们随机变量倾向于什么数值，却不能告诉我们这种趋势要经过多久才会显现，也不能告诉我们随机变量会在平均值附近波动得多剧烈。例如，假设某个六面骰子有三个面标着 1，另外三个面标着 6，那么它的期望值仍然是 3.5，但它“偏离平均值”的程度比普通骰子更大。

那么，如何衡量这种波动的大小？一种做法是测量各结果与 3.5 之间偏差的平均值；但如果保留正负号，这个平均偏差就会是零，因为掷出 1 时的偏差 −2.5 会抵消掷出 6 时的偏差 +2.5。我们也可以取绝对差，不过这样做在实际计算中会遇到麻烦（含绝对值的代数运算比较难处理），而且还存在一些理论上的问题。实践中，人们更倾向于使用“偏差平方的平均值”，并把它称为方差：

$$V(X) = \text{average}((X - EX)^2).$$

由于这里讨论的是统计量，因此所谓“平均”就是求期望，所以：

$$V(X) = E((X - E(X))^2).$$

对于六面骰子，E(X)=3.5，X−E(X) 依次可能为 −2.5、−1.5、−0.5、0.5、1.5、2.5；因此 $(X-E(X))^2$ 依次是 6.25、2.25、0.25、0.25、2.25、6.25。由此，随机变量 X 的方差是 17.5/6。

对方差公式进行代数变形，可以得到另一种有时更方便使用的形式：

$$V(X) = E(X^2) - E(X)^2.$$

期望值和方差具有一些经常使用、在代数上十分方便的性质。例如，假设我们有两个随机变量 X 和 Y，并定义一个新变量 Z=X+Y。那么，E(Z) 是多少呢？结果是：

$$E(Z) = E(X + Y) = E(X) + E(Y).$$

令人惊讶的是，即使 X 和 Y 并非“统计独立”（statistically independent），这个等式仍然成立。对于骰子的例子，这意味着两个骰子的结果可以通过某种方式相互影响。考虑一个极端的例子：我们观察第一个骰子的点数，并让第二个骰子的点数总是与第一个完全相同。即使如此，上面的等式依然成立！这是一个非常强大的性质，程序中经常在不明确说明的情况下使用它。

方差也有类似的性质，但只有当 X 与 Y 相互独立时才成立：

$$V(Z) = V(X + Y) = V(X) + V(Y).$$

下面给出一个反例，说明当 X 和 Y 不独立时，上式并不一定成立。假设先掷出 X，然后让 Y 取这个骰子相对面的点数：如果 X=1，则令 Y=6；如果 X=2，则令 Y=5；如果 X=3，则令 Y=4；依此类推。这样，Z 的取值始终为 7，因此它的方差为零。这显然不是按照两个独立随机变量的方差相加所得到的 2×(17.5/6)。

译注：原书此处有一处明显的变量笔误，写作“X 的方差为 0”；但由于本例中 Z=X+Y 恒等于 7，方差为 0 的应是 Z，而不是 X。上文已按数学含义译为 Z。

方差的一个缺点，是它包含平方运算，因此不够直观。人们经常改用方差的平方根，称为标准差（standard deviation），通常用 σ(X) 表示：

$$\sigma(X) = \sqrt{V(X)}.$$

对于 σ(X+Y)，并不存在同样简洁的计算公式，因此方差具有方便的代数公式这一优点就更加明显了。请注意，在骰子的例子中，标准差为：

$$\sqrt{17.5/6} \approx 1.7.$$

它可以“大致”表示随机变量的值与均值 3.5 之间的平均距离，但并不完全相同，因为真正的平均绝对偏差是 1.5。因此，在实际应用中，把标准差直观地理解为平均绝对偏差通常不会产生太大问题，不过最好始终记得：它们是两个不同的量。

2.11 连续概率（Continuous Probability）

在图形学中，我们经常使用能够在某个数值范围内取值的随机变量。这样的变量通常称为连续随机变量（continuous random variables）。好消息是，离散随机变量的几乎所有内容都可以直接推广到连续随机变量：包括术语、期望值的定义与公式，以及方差的定义与公式。

不过，两者之间存在一个很大的差别：连续随机变量恰好取到任意某个指定数值的概率为零。假设我们有一个在 0 到 10 之间均匀分布的随机变量 X：

```
X = continuous_random_from(-2.3, 10.9).
```

得到 1.7、π 或 e 这些数值的可能性都相同。问题在于，恰好得到 1.7 这个精确数值的概率为零。

好消息是，概率密度函数（density functions）解决了这个问题。就像我们可以用“焦耳每秒”来表示功率一样，在一维情况下，我们可以使用“每单位长度的概率”。在上面的 X=continuous_random_from(-2.3, 10.9) 例子中，我们用来衡量概率的维度是长度。如果长度采用某种未明确指定的单位，而我们只知道数值范围从 0 到 10，那么就会说概率是按“每单位长度”来衡量的。

译注：原书文字先说随机变量的范围是 0 到 10，但随后给出的示例代码却使用 −2.3 到 10.9；最后一句又以 0 到 10 的范围说明单位。中文版保留了原书的这些数值，没有擅自统一。

2.12 蒙特卡洛积分（Monte Carlo Integration）

第 2.5 节讨论了如何“读懂”积分，并将积分抽象为一个 integrate() 函数。但我们究竟该怎样实现这个函数呢？在图形学中，最常用的方法就是蒙特卡洛积分（Monte Carlo Integration）。蒙特卡洛积分涉及的代数推导往往显得繁琐而吓人。不过，如果我们观察下面这个函数：

```
float shade = average(f(), hemisphere)
```

凭借直觉，我们就能找到正确的答案：在半球上随机选取一批点 v_i，分别计算 $f(v_i)$，然后求出这些结果的平均值。例如：

```
float sum = 0.0
N = 10000; // or some other big number the user sets
For (int i = 1 to N)
    vec3 v = random_point_on_hemisphere()
    sum = sum + f(v)
Average = sum / N
```

代码中 N 行的注释表示：也可以采用用户指定的其他较大的采样数量。

事情确实就是这么简单！不过，现在我们需要一个能够在单位半球上随机选取点的函数。最简单的方法是“拒绝采样法”（rejection method）：先在单位球内部均匀选取点。具体做法是，不断在一个单位立方体内均匀随机生成三个数，并检查相应点是否落在单位球内：

```
do
    X = random_from(-1,1)
    Y = random_from(-1,1)
    Z = random_from(-1,1)
while (X^2 + Y^2 + Z^2 > 1)
```

接下来，如果有必要，就把 Z 的符号翻转，使点位于上半球所在的半球体内：

```
If (Z < 0) Z = -Z
```

然后，将该点投影到单位半球的表面：

```
v = unit_vector(X, Y, Z).
```

这样就得到了一种计算平均值的方法。但对于一般的积分，又该怎么办呢？回顾下面的关系：

```
average(f(), domain) = integrate(f(), domain) /
                       integrate(1, domain)
```

因此：

```
integrate(f(), domain) = average(f(), domain) *
                         integrate(1, domain)
```

对于半球而言，integrate(1, domain) 就是半球表面的面积，其值为 2π。

$$\text{integrate}(1, \text{hemisphere}) = 2\pi.$$

因此，蒙特卡洛积分经常可以归结为：对随机采样点上的函数值求平均，再乘以一个常数。这个常数就是积分域的大小，例如长度、面积等。

译注：原书的拒绝采样伪代码在条件判断中将大写 X、Y、Z 写为小写 x、y、z；这里统一为大写。原书第二个 integrate 等式的左侧还多出一个右括号；这里按正确的数学关系排版。

2.12.1 重要性采样（Importance Sampling）

当我们想通过随机采样来求某个函数的平均值，而这个函数的最大值与最小值相差很大时，把采样点集中在某些区域，再利用权重修正采样分布的不均匀性，可能会对我们很有利。概率密度函数（probability density function，PDF）恰好提供了实现这一点所需的工具：如果我们知道一个样本对应的概率密度函数值，就能直接衡量这个区域在多大程度上被“过度采样”了。

因此，如果采用非均匀采样，我们就可以得到：

```
integrate = average_of_nonuniform_samples(f()/p(),
          domain).
```

这个公式很巧妙的一点是：它同样适用于均匀随机采样。在均匀采样的情况下，概率密度函数为：

$$p() = \frac{1}{\text{integrate}(1, \text{domain})}.$$

因此，积分域的“大小”已经包含在概率密度函数（PDF）之中了。

对于任何一个具体的蒙特卡洛重要性采样问题，至少在开始时，我们都可以遵循一套相当固定的步骤：

1. 明确函数 f() 是什么，以及积分域是什么（例如，单位球面上的点，或某个三角形上的点）。
2. 选择一种在积分域上生成随机样本 x_i 的方法，并确保能够为每个样本计算其概率密度函数值 $p(x_i)$。
3. 对许多个样本 x_i，计算比值 $f(x_i)/p(x_i)$，然后对这些比值求平均。得到的结果就是我们对积分的估计。

另一个巧妙之处在于：我们可以使用任意一个 p()，而最终仍会收敛到正确答案（但有一个前提：在 f() 不为零的地方，p() 也必须不为零）。选用哪一个 p()，只会影响估计值收敛的速度。因此，在调试代码时，我们通常先从常数形式的 p() 开始。

常见问题（Frequently Asked Questions）

- 为什么没有向量除法?

事实证明，对于向量，并不存在一种能够与普通除法形成“漂亮”类比的运算。不过，如果深入研究这个问题，就可以由此引出四元数（quaternions）的概念（参见本章注释中提到的 Hoffmann 的著作）。

- 对于边数超过三条的多边形，有没有像重心坐标那样简洁的坐标系统?

很遗憾，没有。即使是凸四边形，情况也要复杂得多。这也是三角形在图形学中如此常见、被广泛用作基本几何图元的原因之一。

- 三维直线有隐式表示形式吗？

没有。不过，两个三维平面的交集可以确定一条三维直线，因此我们可以用两个同时成立的三维隐式方程来描述一条三维直线。

- 准蒙特卡洛（quasi-Monte Carlo，QMC）或蓝噪声采样，与蒙特卡洛采样有什么关系？

蒙特卡洛方法的核心思想是：对大量“公平”的样本求平均，以估计真实的平均值。这里的“公平”可以从统计学意义上定义。但对于一些样本集合，即使它们并不是随机生成的，我们也能够证明它们同样是“公平”的。

准蒙特卡洛样本集合就是其中一类。这类样本具有明显的确定性结构，并非随机样本；但它们在一种非统计学的、形式化的意义上具有均匀性。与随机样本相比，这类集合往往能改善收敛速度。

蓝噪声样本集合则为采样点加入了一些约束，以避免样本聚集成团。与准蒙特卡洛样本一样，蓝噪声样本即使不是完全随机的，也可能改善收敛速度。

在实际应用中，大多数技术最初都使用蒙特卡洛的数学形式来推导，因为这样更容易处理数学问题。然后，开发者再把准蒙特卡洛或蓝噪声采样点替换到代码中，并凭借经验确信：在实际使用中，采样点足够均匀就已经满足需要。

注释（Notes）

向量分析（vector analysis）的历史尤其有趣。它在很大程度上由格拉斯曼（Grassmann）于 19 世纪中叶创立，但当时并未受到重视，后来又被其他人重新发明（Crowe, 1994）。如今，在图形学领域，格拉斯曼拥有一批追随其思想的研究者，他们正基于格拉斯曼的某些观点发展几何代数（Geometric Algebra）（Doran & Lasenby, 2003）。

如果读者想知道，为什么我们使用的特定标量积与向量积在某种意义上是恰当的运算，以及为什么没有一种被普遍采用的向量除法，那么读一读简明的《论向量》（About Vectors，Hoffmann, 1975）会很有启发。

另一项重要的几何工具是四元数（quaternion），它由汉密尔顿（Hamilton）于 19 世纪中叶发明。四元数在很多情况下都很有用，尤其适合处理与朝向（orientation）有关的问题（Hanson, 2005）。

习题（Exercises）

1. 集合的基数（cardinality）是指这个集合包含的元素个数。采用 IEEE 浮点数表示方法（见第 1.5 节）时，浮点数集合的基数是多少？
2. 是否能够实现一个将 32 位整数映射为 64 位整数、并且具有明确定义的逆函数的函数？是否所有从 32 位整数到 64 位整数的函数，都具有明确定义的逆函数？
3. 使用三个区间的笛卡尔积来表示单位立方体（即 x、y、z 坐标都在 0 到 1 之间，且包括两个端点）。

4. 如果可以使用自然对数函数 ln(x)，请说明如何利用它实现以 b 为底的对数函数 log(b,x)。当 b 为负数时，这个函数应该如何处理？假设采用 IEEE 浮点数实现。

5. 求解一元二次方程 $2x^2+6x+4=0$。

6. 实现一个函数：输入一元二次方程 $Ax^2+Bx+C=0$ 的系数 A、B、C，并计算它的两个解。该函数应返回有效解（即不是 NaN 的解）的个数，并通过输出参数填写解，使两个解中较小的那个排在前面。

7. 证明第 17 页给出的一元二次方程求根公式的两种形式是等价的（假定运算完全精确）；并解释计算每一个根时，应怎样选择所用的公式，以避免对两个几乎相等的浮点数做减法，从而造成精度损失。

8. 对于三维向量 a、b、c，用一个反例证明：$a\times(b\times c)=(a\times b)\times c$ 并不总是成立。

9. 给定两个不平行的三维向量 a 和 b，计算一个右手正交归一基，使 u 与 a 平行，而 v 位于由 a 和 b 确定的平面内。

10. 函数 $f(x,y,z)=x^2+y-3z^3$ 的梯度是什么？

11. 与坐标轴对齐的二维椭圆，其参数形式是什么？

12. 经过三维点 (1,0,0)、(0,1,0)、(0,0,1) 的平面，其隐式方程是什么？参数方程是什么？这个平面的法向量是什么？

13. 给定四个二维点 a_0、a_1、b_0 和 b_1，设计一个稳健的过程，判断线段 a_0a_1 与线段 b_0b_1 是否相交。

14. 设计一个稳健的过程，计算某个二维点相对于三个不共线二维点的重心坐标。

15. 计算初等微积分中的各种一维积分，并改变采样点的数量。随着采样数量增加，计算结果的收敛速度有多快？